

HoneyLLMd: A Large Language Model-Powered Adaptive Honeytrap System

Wenjun Fan , *Member, IEEE*, Fei Du , Zhihui Wang , Qiyu Cao , Lekun Liu , Kejing Chang ,
and Sang-Yoon Chang 

Abstract—A honeypot is a cybersecurity mechanism designed to lure attackers by emulating vulnerabilities and serving as a decoy, intentionally crafted to detect and analyze malicious activities. However, achieving a balance between the ability to engage attackers extensively and the need to minimize exposure of system resources remains an enduring challenge. High-interaction honeypots (HIHs) provide in-depth system-level observation through full operating systems, yet they inherently increase the risk of being compromised due to the extensive exposure of functionalities. In contrast, low- and medium-interaction honeypots (LIHs/MIHs) emulate only minimal or disguised services, which cannot spawn reverse shells or execute real command responses, but significantly reduce the risk of system intrusion. To overcome this limitation, we present a large language model (LLM)-powered adaptive honeypot system capable of supporting both bind and reverse shell interactions with authentic shell-like responses generated by the LLM rather than a real OS. This design enables convincing, continuous “request–response” interactions in which attackers unintentionally reveal behavioral and intent data. Furthermore, a hierarchical probabilistic automaton (HPA) is employed to model attack transitions and dynamically adapt honeypot strategies. Experimental validation on real-world attacks demonstrates that the proposed system achieves superior engagement and data capture while maintaining low security risk.

Index Terms—Deceptive defense, honeypot, large language model, hierarchical probabilistic automaton, authentic response.

I. INTRODUCTION

A HONEYPOT is a trap tool [1] deliberately created to be probed, attacked and compromised in order to capture

Received 18 October 2025; revised 26 March 2026; accepted 6 April 2026.
Date of publication 15 April 2026; date of current version 28 April 2026.
Recommended for acceptance by Dr. Weiting Zhang. (*Corresponding author: Fei Du.*)

Wenjun Fan is with the School of Engineering and Technology, University of Washington, Tacoma, WA 98402-3100 USA (e-mail: wjfan@uw.edu).

Fei Du is with the Department of Computer Science, University of Liverpool, L69 7ZX Liverpool, U.K., and also with the School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou 215123, China (e-mail: Fei.Du@liverpool.ac.uk).

Zhihui Wang is with the Department of Electrical and Systems Engineering, University of Pennsylvania, Philadelphia, PA 19104 USA (e-mail: wang52@seas.upenn.edu).

Qiyu Cao is with the School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou 215123, China (e-mail: Qiyu.Cao22@student.xjtlu.edu.cn).

Lekun Liu is with the School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou 215123, China (e-mail: Lekun.Liu22@student.xjtlu.edu.cn).

Kejing Chang is with the School of Computing, National University of Singapore, Singapore 117417 (e-mail: e1520128@u.nus.edu).

Sang-Yoon Chang is with the Computer Science Department, University of Colorado, Colorado Springs, CO 80918 USA (e-mail: schang2@uccs.edu).

Digital Object Identifier 10.1109/TNSE.2026.3683862

malicious behaviors. As honeypots have no production value, the egress traffic from a honeypot can indicate that the honeypot has been compromised. Over the past decade or so, the development of honeypot projects and research has been greatly affected by the evolution of the virtualization technology [2], since virtualization can facilitate the creation and deployment of the honeypot efficiently, which results in the rapid growth of virtual honeypot solutions as well [3], [4]. Taking advantage of virtualization technology, virtual honeypots support large-scale deployment and dynamic configuration. However, other than the honeypots that use the genuine guest operation system (OS) as the decoy for attracting the network adversary, virtual honeypots do not often provide the entire system functionality and only use the minimal system resource. Usually, we call the virtual honeypot using minimal or limited system resources low-/medium-interaction honeypot (LIH/MIH), which exposes low-security risk to the adversary but it lacks the fidelity to interact with the adversary to learn the adversary's system-level activity. By contrast, the virtual honeypot building on the genuine OS is called a high-interaction honeypot (HIH), which has high fidelity but is less scalable for deployment, unless hybrid system architecture is involved where the lightweight and distributed front-ends (e.g., sensors or LIHs) are used to capture data in large-scale, e.g., Collapsar [5] and Potemkin [6]. However, such hybrid systems still suffer the risk of being entirely compromised by the network adversary in terms of exploiting the vulnerability of the OS. Further, we notice that most advanced attacks focus on obtaining a shell with the compromised system to further command and control (C&C) it. Such compromised systems can constitute a botnet to carry out various attacks such as distributed denial-of-service (DDoS), which is still at the top of the list of current cyber threats [7].

The balance between security and utility of honeypot is an arduous research challenge. HIH is able to execute the shell payload to further interplay with the adversary, however, it often brings the full compromise risk and lacks the means to control the shell TCP connection. On the other hand, LIH/MIH has little risk to get compromised, but it is neither to run any payload for creating a reverse shell for the attacker to undertake C&C due to the minimal system functionality nor to provide real shell command responses.

To this end, this paper aims to leverage the large language model (LLM) [8], [9] to generate authentic-seeming, yet fabricated responses tailored to adversarial inputs such as shell-codes, commands, keystrokes, etc. LLMs are sophisticated AI systems capable of comprehending and generating human

language through extensive text data processing. Consequently, LLM-based honeypots can establish highly realistic interactions with adversaries. Rather than requiring the deployment of a fully operational system, the LLM-powered honeypot can convincingly mimic one by generating appropriate command outputs. This approach offers significant cost and security advantages over conventional HIH, while maintaining a higher level of credibility compared to LIH/MIH. Furthermore, our approach transcends simple LLM-based emulation by integrating hierarchical probabilistic automaton (HPA)-based strategy that governs the interaction logic. By modeling the attack process through an HPA, the honeypot gains an advanced intelligence: it can predict the attacker's next command and proactively steer the interaction toward low-payoff attack paths. This maneuver ensures that the LLM generates more appealing and contextually precise responses, which misleads the attacker into believing that the exploitation was successful, thereby inducing further novel malicious activities. The contributions of this paper can be summarized as follows:

- A comprehensive literature review is delivered to identify the development tendency and current problems in honeypot systems.
- A large language model-powered adaptive honeypot termed HoneyLLMd is proposed, which leverages the hierarchical probabilistic automaton to model the attack process in order to adapt the strategy for responding the attack activities.
- Both bind shell and reverse shell have been realized, and a suite of evaluation metrics for shell performance is introduced, aimed at assessing the responses of shells utilizing various LLMs.
- A comparative analysis of several prominent large language models (like GPT-5, GPT-4o-mini, Gemini-2.5-flash, and Claude-haiku-3.5) is conducted using a unified benchmark.
- A proof-of-concept system has been developed and deployed to detect real-world attacks. Following this, a series of experiments were carried out to assess the effectiveness of the proposed method.

Ethics Statement: This research adheres to ethical principles and guidelines in conducting and reporting our findings. All data collection and analysis were carried out with due consideration for the ethical implications.

Paper Organization: Section II provides a comprehensive literature review of honeypot systems, and outlines the research problems. Section III proposes the design of the LLM-powered adaptive honeypot. Section IV details a proof-of-concept and its deployment for capturing real-world attacks. Section V presents the experimental results and discusses the limitations. Section VI draws a conclusion and outlines potential avenues for future research.

II. STATE-OF-THE-ART

This section reviews research on honeypot systems, with a focus on honeypot interaction levels, honeypot shells, and intelligent honeypots, including those based on statistical methods,

machine learning, and LLM-driven approaches. Finally, the problem statements which this paper builds on are provided.

A. Honeypot Interaction Level

The interaction capacity of a honeypot is generally defined as the extent to which the information system resources are exposed to network adversary. Therefore, honeypot systems can be classified into several types as follows.

1) *Low-Interaction Honeypots (LIH):* An LIH usually exposes less access to the attackers, emulating basic protocols and network services of an OS. Virtual environments are often employed in the LIH to simulate the functionality. For example, Honeyd [10] is a seminal virtual honeypot framework that simulates entire network topologies at the network layer. By emulating the TCP/IP stack of various OSs through configurable 'personalities', it can effectively deceive remote fingerprinting tools like Nmap. The main objective of LIHs is to detect and collect malicious activities, such as unauthorized probes or login actions [11]. The development of LIH has expanded with the scope of cyber attacks, such as HoneydV6 [12] that simulates entire IPv6 networks on a single host, KFSensor [13] for aid LIH in fingerprinting attack, and HoneyPy [14] for minimizing Slowloris attack. Beyond these examples, LIHs are primarily valued for their resource efficiency and ability to be deployed at scale, serving as an early-warning sensor network. However, their inherent limitation lies in the shallow level of emulation, which can be detected by sophisticated attackers.

2) *Medium-Interaction Honeypots (MIH):* MIHs offer a higher degree of service emulation than their low-interaction counterparts. They are capable of limited interaction with attackers, going beyond simple traffic identification, all while operating without a real underlying OS. Representative examples such as Amun [15] and Dionaea [16], which emulate vulnerable services by pre-defining the corresponding service reply logic to capture attack payloads. Cowrie [17] under a shell mode implements a UNIX-like shell environment, providing a deceptive shell to log adversary commands and keystrokes. While MIHs offer deeper interaction than LIHs-enabling behavioral analysis of attack sequences, they remain constrained in capturing full intrusion chains, such as post-exploitation activities including rootkit installation.

3) *High-Interaction Honeypots (HIH):* HIHs are characterized by their provision of a realistic, compromisable environment for adversaries. This is most commonly achieved by deploying real OS and applications, allowing attackers to perform full intrusion chains. Alternatively, HIHs can be implemented through high-fidelity emulation (e.g., using instrumented full-system virtual machines (VMs)). The honeypot fidelity comes with increased risk, as attackers gain actual system access [2]. In other words, if the honeypot would like to learn more from the adversary, it needs to provide more resources for the malicious interaction, while that also brings more threats and security concerns. For example, Argos [18] used QUME to emulate the OS, and exposed complete system resources to attackers; however, it faced challenges in security maintenance, dynamic reconfiguration, and scalable deployment. Cowrie in proxy

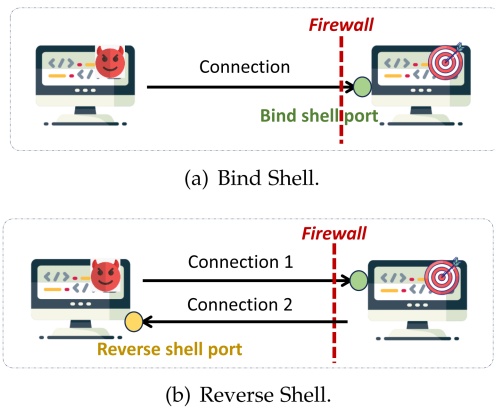


Fig. 1. Honeypot Shell Description.

mode [19] can redirect interactions to fully functional backend VMs, transforming it into an HIIH. In addition, containerization technology is also often employed in honeypot system with separated host access. HoneyKube [20] developed by Gupta et al. is a microservice-based network honeypot that achieves maximum control over system vulnerabilities and permissions, as well as increases the number of interactions. The application scenarios of HIIH are diverse, such as HoneyPLC [21] for Industrial Control Systems (ICS) and for specific companies scenarios [22]. In summary, while HIIHs provide unparalleled insight into attack behaviors, they require significant resources and robust isolation mechanisms to mitigate security risks.

4) *Hybrid Honeybots (HyH)*: HyHs integrate multiple honeypot types to balance scalability and interaction depth. For instance, BitSauce [23] combines LIHs and HIIHs, using lightweight LIHs for broad monitoring and invoking HIIHs only when sophisticated interaction is required, thereby optimizing resource utilization. Alberdi et al. [24] proposed a transparent redirection mechanism that stealthily channels malicious traffic to appropriate honeypots while shielding production systems. Similarly, [25] integrated LIHs and HIIHs to address the trade-off between coverage breadth and behavioral fidelity. Furthermore, in order to eliminate the situation of identity exposure caused by the lack of interaction with other network nodes in a single honeypot within the network traffic, honeynet was introduced [26], [27]. T-Pot [28] is an open-source platform that creates a dedicated segment to deploy LIHs and HIIHs to track hackers on either a single host or across multiple systems. However, the security risk of honeynet remains, as they hold large attack surface, complicate configurations, and high maintenance cost [26].

B. Honeypot Shell

Network adversaries commonly establish remote control over the compromised hosts by spawning interactive command shells, which serve as C&C channels. These shells are broadly classified into two types: bind and reverse shells.

In a **bind shell** (Fig. 1(a)), the compromised host listens on a specified network port, awaiting an incoming connection from the attacker. This method succeeds only if the target environment

permits inbound connections to the chosen port. In contrast, a **reverse shell** (Fig. 1(b)) involves the compromised host initiating an outbound connection to a listener controlled by the adversary. Compared to bind shell, this approach offers several operational advantages for adversaries [29], [30]:

- 1) Outbound traffic is typically subject to fewer firewall restrictions than inbound connections;
- 2) Reverse shells circumvent the need to determine the target's public IP address, thus avoiding NAT traversal complications;
- 3) Outbound callbacks can maintain stealth over extended periods, evading detection more effectively than persistent listening services.

These advantages have established reverse shells as the dominant C&C pattern in modern exploitation frameworks.

However, conventional honeypots—particularly LIHs and MIHs—are inherently limited in their ability to handle reverse shells. For instance, the LIH Honeyd [10] and the MIH Cowrie [17] support only bind shell establishment. Although HIIHs that host full OS can, in theory, support reverse shells, their operational complexity and security risks limit widespread deployment. This is particularly concerning given the prevalence of reverse shells in mainstream penetration frameworks. For instance, Metasploit [31] includes a majority of payloads dedicated to establishing reverse shells post-exploitation, with numerous modules supporting raw TCP and HTTPS-based callbacks. Moreover, attackers increasingly employ covert channels, such as DNS tunneling [32], WebSocket-based callbacks [33], and ICMP tunneling [34] to evade conventional network controls.

The widespread adoption of reverse shells for C&C, coupled with their limited support in conventional honeypots, motivates the development of honeypot systems capable of safely spawning and monitoring reverse-shell connections to enhance threat intelligence collection.

C. Honeypot Intelligence

1) *Statistic Approach*: To counter increasingly sophisticated anti-honeypot techniques, traditional honeypots have become insufficient to effectively deceive attackers. In this context, statistic-based honeypots employ mathematical abstraction to capture the stochastic nature of attacker behaviors. These models aim to quantify uncertainties, optimize defensive responses, and support decision-making under incomplete information.

Game theory is particularly relevant to the modeling and analysis of strategic operations of information [35], including deception tactics that are highly applicable to honeypots. In [36], the authors proposed a game-theoretic honeypot framework integrated with an HPA. This framework models the interactions between HIIHs and attackers, as well as the command transition probability. The Monte Carlo simulations were tested to find the Nash equilibria, with the goal of maximizing the honeypot's utility. However, the efficacy of such model heavily relies on the assumption of rational adversaries and predefined behavioral patterns, limiting their effectiveness against non-rational, automated, or novel threats. An innovative pseudo honeypot game (PHG) [37] has been proposed, which combines Bayesian

networks to infer the attacker's intentions, enhances deceptiveness and prevents DDoS attacks. Moreover, to address the limitation of traditional game theory on dynamic behavior modeling, stochastic games (SG) was adopted. In [38], the honeypot strategy was optimized in terms of the uncertainty of network and opponents based on one-sided partially observable stochastic game (OS-POSG). SG was also employed for honeypot defense in Uncrewed Aerial Vehicle (UAV) systems [39]. In addition, Markov Decision Process (MDP) and its variants offer another principled methodology. In paper [40], MDP was utilized to model the interactions between botmasters and honeypots, and Partially Observable Markov Decision Process (POMDP) was extended to help honeypot predict uncertain honeypot states as determined by botmasters. Meanwhile, Tapaswi et al. [41] deployed a Markov chain-based honeypot model on LIHs and HIHs to identify systems most likely to be attacked, dynamically changing honeypot IP addresses to avoid detection and collect more attack data.

Nevertheless, statistical models often struggle with the complexity and adaptability of real-world threats. A significant limitation remains their inability to dynamically enhance the fidelity of the honeypot's interactions.

2) *Machine Learning-Based Approach*: In response to the shortcomings of statistical methods, honeypots have begun incorporating learning-based methods.

ML methods have been widely used to assist honeypot systems in malicious traffic detection and post-attack data analysis [42]. In [43], Natural Language Processing (NLP) for detecting social engineering (SE) attacks was developed with labeled SE data for honeypot decoys generation. TwinPot proposed by [44] integrated ten different classification algorithms to detect internal attacks. Holgado et al. [45] proposed a multi-step attack prediction method based on Hidden Markov Model (HMM), and Li et al. [46] improved this method by using HMM and Long Short-Term Memory (LSTM) with honeypot captured data. Fraunholz et al. [26] took advantage of ML to identify unique types of attack sessions and characteristics based on issued commands and provided credentials. HuntGPT [47] introduced Explainable AI (XAI) to ML techniques for reliable and actionable intrusion detection. Moreover, ML for anti-detection is realized in [48], which developed an learning-evaluated honeypot using Reinforcement Learning (RL), increasing the number of instructions in the "secure shell" sequence presented by the honeypot, thereby increasing the number of interactions and reviewing more attack information.

Also, the configuration of the honeypot system has been implemented in a ML manner. Taofeek et al. [49] proposed a cognitive deception model (CDM) using a neuro-semantic model to generate honeyfiles to deceive the adversaries. Paper [50] proposed a method that can identify, analyze, and cluster the machines and equipment entities in the network to realize the dynamic configuration, deployment and maintenance strategy of honeypot. The study by Franco et al. [51] proposed S-pot, an intelligent honeypot capable of learning from captured attacks to dynamically adjust Software-Defined Networking (SDN) configurations.

In addition, ML is able to optimize the deceptive response performance. Honey-imprint [43] employed Generative Adversarial Networks (GAN) for generating deceptive decoys, and steganography for imprinting the honey watermark. However, this work is limited in its inability to generate real-time fake responses for deceiving SE attackers. In [52], an intelligent interaction honeypot named IoTcandyJar was realized. Specifically, the behaviors of different types of IoT devices are obtained and learned by a variety of heuristics and ML mechanisms to generate credible responses. HoneyIoT presented by [53] modeled the attacker's behavior using RL and MDP, and used differential analysis to provide real time and high fidelity responses. Chamotra et al. [54] presented SAGE, a novel IoT honeypot that leverages a Finite-State Machine (FSM) for protocol emulation, and augments it with a GraphRAG-LLM framework. This architecture dynamically generates context-aware, deceptive responses, mitigating fingerprinting and engaging adversaries proactively. Chamotra et al. [54] presented SAGE, a novel IoT honeypot that leverages a Finite-State Machine (FSM) for protocol emulation, and augments it with a GraphRAG-LLM framework. This architecture dynamically generates context-aware, deceptive responses, mitigating fingerprinting and engaging adversaries proactively.

ML techniques have made great contributions to honeypot systems. Yet now the rise of LLMs represents a new frontier, promising even greater transformative power.

3) *LLM-Based Approach*: LLMs represent a specific category within the broader group of pre-trained language models (PLMs). The landscape of high-performance LLMs has experienced a notable expansion, particularly after OpenAI unveiled ChatGpt [55]. These models have showcased exceptional capabilities across various fields [56], [57] with significant contributions to cybersecurity, including areas like code analysis [58], vulnerability remediation [59], generation of control code [60], reverse engineering [61], and protocol fuzzing [62]. Furthermore, the advent of Generative Pre-trained Transformer (GPT) models by OpenAI and other models such as Deekseek, Gemini, and Claude have positioned LLMs as powerful mechanisms for automating tasks related to vulnerability exploitation [63], [64], [65] and penetration testing [66]. The potential of GPT in generating cybersecurity policies to deter and mitigate ransomware attacks was investigated by [67]. Their research demonstrated that GPT-generated policies have the potential to outperform human-generated policies in certain contexts, particularly when optimized with tailored input prompts. However, human input remains crucial in guiding the generation process. In [68], a tool for helping security analyst was proposed, which can produce a swift and effective response against incoming threats. While the tool can generate a ranked list of potential solutions for the analyst to quickly assess, human intervention is still required to implement and evaluate the appropriate response.

LLM agents are advanced systems that leverage LLMs to address problems, devise solutions, and execute these solutions using a suite of tools [63]. In the context of cybersecurity, LLM agents must incorporate three essential components to function effectively. First, they need the capability to interact with tools and APIs [69], [70], [71] enabling autonomous actions that

would otherwise require human involvement, with responses integrated as contextual feedback. Second, they must be adept at planning and responding to tool and API outputs [69], ranging from straightforward tasks such as incorporating outputs into the model for further context, to more complex planning processes. Finally, the ability to read and analyze documents (closely related to Retrieval-Augmented Generation, RAG) [72] is crucial, as it allows the agent to focus on pertinent topics and refine its problem-solving capabilities. At present, API calls are the mainstream approach for integrating LLMs into honeypot systems. Local deployment is less common due to its prohibitively high resource demands and maintenance costs. Nevertheless, the latency and instability inherent to API-based solutions present significant concerns [73].

Given the autonomous planning and tool-integration capabilities of LLM agents, researchers have begun to explore their application in honeypot systems. Initially, LLMs have been widely adopted for analyzing honeypot data [74]. For instance, GPT-2C [75] fine-tuned an LLM to promptly interpret Cowrie logs, delivering near-real-time threat analysis that can support Intrusion Detection System (IDS). Although GPT-2 is mainly used as a pre-trained transformer, this work has not yet generated real-time fake responses. [76] employed NLP to evaluate and cluster different types of cyber attack against SSH service. In [77], a dialogue agent named CYGENT was designed to assist with log file summarizing and analyzing.

Subsequent work extended LLMs from passive analysts to active generators of deceptive decoys: Raví et al. [78] used natural-language models to create “honeypot files” (fabricated lure documents) that prolong attacker engagement, and Reti et al. [79] employed multiple LLMs to synthesize diverse “honeytokens” (e.g., log entries, database records, and simulated vulnerable services) as part of a comprehensive cyber-deception solution. The ability of LLMs to build entire honeypot shells was also being explored. In [80], shellLM, an LLM-supported fake conversational shell, was first explored, which aimed to address the drawbacks in the previous deception approaches, such as deterministic responses and a lack of adaptability, etc. Based on shellLM, an extensible and modular shell architecture LIM-BOSH was also developed by [81].

LLMs for deceptive responses generation in terms of adaptive interaction and semantic realism have been extensively studied recently. ShellGPT [82] demonstrated LLM’s superiority compared to other baseline models for shell language understanding tasks. A honeypot called DecoyPot [83], which combines RAG and LLMs has been presented. It generates more realistic API-like responses dynamically based on the received Web API query requests. Paper [84] proposed a honeypot supported by GPT-3, enabling human-like conversations with attackers, thereby providing a more realistic and engaging environment. HoneyGPT [85] was presented, which mainly focused on enhancing the shell response capabilities of Cowrie by integrating the OpenAI GPT-3.5-Turbo and GPT-4. [86] leveraged fine-tuned open-source LLMs to develop an interactive honeypot, which significantly enhances realism and effectiveness in deployment. The HoneyLLMs proposed by [87] and [88] concentrate on enhancing authentic bind shell responses using various LLMs.

TABLE I
COMPARISON OF EXISTING HONEYPOT INSTANCES. THE LISTED HONEYPOT INSTANCES ARE PRESENTED IN CHRONOLOGICAL ORDER. THE HONEYPOTS MARKED BY INDICATE THAT THEY HOLD THE CORRESPONDING CHARACTERISTIC OR TECHNIQUE.

Honeypot	Interaction				Reverse	Tech			Authentic Response	Real time Response
	LIH	MIH	HIH	HyH		Statistic	ML	LLM		
Honeyd [10]	•									•
[25]				•	•				•	
Argos [18]			•		•	•			•	
Shark [24]			•		•				•	
[36]			•			•			•	
Amun [15]		•								•
BitSaucer [23]				•					•	
HoneydV6 [12]	•				•					
[41]				•		•				
Cowrie [17]		•	•		•					•
[50]	•									
IoT CandyJar	•					•	•		•	
[52]							•			
[26]		•					•			
[37]		•				•	•			•
KFSensor [13]	•					•				
Dionaea [16]		•								•
[48]		•				•	•			
HoneyPy [14]	•									
[22]			•	•						
HoneyPLC [21]			•	•				•	•	
GPT-2C [75]		•						•		
S-pot [51]		•						•		
Honey-imprint		•						•		
[43]										
[39]		•				•	•			•
HoneyIoT [53]			•		•	•	•		•	•
HoneyKube [20]			•						•	•
TwinPot [44]		•				•	•			
ShellLM [80]	•							•	•	
[84]			•					•		
LIMBOSH [81]		•					•	•		
HoneyGPT [85]		•				•				•
HoneyLLM (Fan et al.) [87]		•						•	•	•
HoneyLLM (Guan et al.) [88]	•							•	•	
LLM Honeypot [86]		•					•	•	•	
OHRA [90]				•	•			•	•	
DecoyPot [83]		•					•	•	•	
GASH [89]		•					•	•	•	
SAGE [53]		•				•	•	•	•	•
HoneyLLMd (ours)	•				•	•	•	•	•	•

Prasad et al. [89] combined RL and generative AI to enhance the honeypot replies for the simulated SSH service. Safargalieva et al. [90] introduced “Ohra”, a scalable honeypot framework built on LLMs. The system employs multiple protocol models and utilizes prompt control alongside session management to produce context-aware responses.

D. Problem Statement

Table I statistics the aforementioned honeypot instances with their characteristics. When the honeypot can respond to commands in the same way as a real system does, we consider it to be able to provide “authentic responses”. When the response delay aligns with the actual situation, it conforms to the characteristics of “real-time response”. It should be noted that some systems can offer real-time responses, but the content may not be authentic.

From Table I, several drawbacks of current honeypot systems can be summarized as follows:

- Existing honeypots are limited to passive request-response behavior, simply replying when possible without adaptive or intelligent strategies to actively engage, manipulate, or prolong attacker interactions.
- Reverse shell capability mainly appears in HIHs and some HyHs, while most LIHs and MIHs remain restricted to bind interactions. In addition, while ML and LLM techniques

have been introduced and integrated to honeypot systems, the reverse shell functionality is still underrepresented.

- Most honeypots are unable to achieve the simultaneous realization of authentic response and real-time response. Majority of LLM-powered honeypots can achieve high linguistic realism but limited to real time reply for maintaining deception credibility.

Therefore, our work is motivated to facilitate both bind and reverse shell capabilities, enabling effective responses to various attack scenarios. Also, it is crafted to emulate multiple OS, enhancing its versatility. Moreover, our design transcends the confines of OpenAI's models, boasting compatibility with a broad spectrum of LLMs. We have also dedicated considerable effort to fine-tuning the base model, as well as probability and separated state modeled automaton, aiming to generate more realistic and interactive shell interactions. In essence, although previous studies have explored honeypot systems and deception methodologies, our innovative application of LLMs within this domain marks a significant and novel advancement.

III. DESIGN OF HONEYLLMD

This section outlines the threat model underlying our approach, as well as the design of the adaptive HoneyLLMd. It includes an overview of the system architecture, a detailed description of each functional module, the honeypot modeling approach, and the evaluation metrics used.

A. Threat Model

The proposed HoneyLLMd is designed to enhance traditional low-/medium-interaction honeypots by integrating LLM to generate imitative shell responses. Our work focuses on engaging and mesmerizing attack activities through shell interaction based on intelligence strategies. Thus, in addition to the basic data capture capabilities for port scanning, probing, traffic flooding, etc., our HoneyLLMd is capable of maneuvering to resist handcrafted/script-generated shell commands. It is assumed that the adversary is able to exploit the vulnerabilities set up by our honeypot and can create bind/reverse shell as if our honeypot can recognize the exploit payload and spawn the corresponding shell session with the attacker.

Also, we stress on the attackers seeking targets of opportunities (like internet-based opportunists), however, they are not limited to the attacker aiming for targets of choice, e.g., target intruders in an enterprise network. Essentially, it depends on how the HoneyLLMd would be used and where it will be deployed. We assume that it would be more likely to be deployed in the Demilitarized Zone (DMZ), while there is no constraint to deploy it inside networks. Furthermore, the adversary may employ honeypot detection techniques based on side-channel signals such as latency, IP addresses, and traffic patterns, whereas our deception defense approach primarily focuses on generating authentic shell responses rather than manipulating other side-channel signals. Thus, enhancing deception quality by tweaking side-channel signals is outside the scope of this threat model.

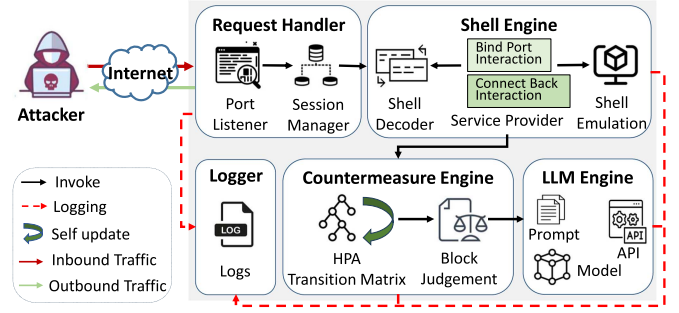


Fig. 2. An Overview of the Design of HoneyLLMd.

B. System Overview

Fig. 2 illustrates the design of the proposed LLM-powered adaptive honeypot. The entire system is composed of four components—Request Handler (RH), Shell Engine (SE), Logger, and Countermeasure Engine (CE)—and one external component—the LLM Engine (LE). HoneyLLMd not only provides bind shell functionality but also introduces support for reverse shell capabilities.

The *RH*, as its name suggests, is responsible for handling malicious connection requests, particularly managing concurrent requests, processing payload operations, and invoking the appropriate shell emulation processes. Within *RH*, the Port Listener monitors traditional bind shell ports as well as other ports that may receive malicious payloads, acting as a preliminary filter to facilitate reverse shell functionality. Additionally, *RH* forwards and displays the commands and responses exchanged between the attacker and the shell.

The *SE* primarily provides deceptive responses to attackers using LLM engine, including fake shells for traditional bind shell requests and reverse shells for reverse connection requests identified from malicious payloads.

The *Logger* records and stores content generated from all modules of HoneyLLMd, covering essential details such as incoming commands, decoding procedure, block decisions, and LLM responses for system analysis and optimization.

The *CE* consists of two modules. The first part is the HPA, which is constructed by preprocessing the raw incoming commands into states, and the transition matrices which deliver the frequency and probability transitions between states. They are continuously updated when detecting new transitions. The another part is Block Judgement, which determines whether incoming content should be blocked according to the transition matrix.

The *LE* is an external module that the virtual honeypot relies on. Multiple LLMs can be integrated into the *LE* to meet the performance requirements necessary for effective deception and defense.

C. Key Functional Components

In terms of the systematic description of the whole workflow, each key component is detailed as follows.

1) *Request Handler (RH)*: This component is comprised of two programs: port listener and session management.

The **Port Listener** is responsible for monitoring specific ports on the honeypot system. In addition to the SSH service on port 22, it also listens on other ports to simulate different service types and attract a broader range of attackers attempting various attack methods. This allows the system to capture malicious payloads that can be used to prepare for subsequent reverse shell interactions. By monitoring these ports, the honeypot system can capture a wider array of attack behaviors, analyze different types of threats, and enhance the system's ability to respond to diverse attacks. Once a connection request is detected, it triggers the session manager to handle the incoming connection. This component ensures that attackers can connect to the honeypot system and proceed to the next stage of interaction.

The **Session Manager** handles both the transport-layer connection and the application-layer session. Specifically, once the TCP three-way handshake is completed, this program can provide application-layer-specific responses according to the request, such as setting up an SSH session. When the remote attacker closes the connection or the program detects that the process has existed, it will terminate the corresponding process. Further, the Session Manager is in charge of ensuring the session input piped into `stdin`, while `stdout` redirected to the session output. In our design, the Session Manager process is additionally responsible for passing the attacker's C&C requests to the Shell Emulation and forwarding the bogus responses generated by the Shell Emulation back to the attacker.

2) **Shell Engine (SE)**: The Shell Engine is the core component responsible for simulating the shell environment and interacting with the attacker. It consists of several programs that work together to create a convincing interaction.

The **Shell Decoder** is used to dissect the malicious payload in order to obtain the parameters, especially for generating the reverse shell. We employ two methods for decoding the shellcode. One relies on the regular expression to match the shellcode patterns, and the other uses LLM to analyze the shellcode and get parameters.

The **Bind Port Interaction** handles direct shell interaction by binding to a local port (i.e., 22 for SSH and 23 for Telnet). In this mode, the attacker connects to a specific port on the honeypot, where the Shell Emulation processes incoming commands and returns appropriate fake responses. The **Connect Back Interaction** prepares to initiate a reverse connection upon notification from the Decoder, typically triggered after an attacker's malicious payload has been injected into the system. The attackers believe they have successfully exploited a system vulnerability and gained shell access, while the Shell Engine initiates a reverse connection back to the attacker's machine. This method further entices the attacker to continue their operations, making them believe they have full control over the system.

The **Shell Emulation** program takes the critical responsibility for preparing the execution environment, including environment variables, arguments, standard input/output (I/O), and then executing the command. Regarding the command execution, it essentially invokes the LLM API to obtain the authentic response and then forward the response to the Session Management which further returns the bogus response to the attacker.

3) **LLM Engine (LE)**: The LLM Engine plays a decisive role for the HoneyLLMd. This external component includes the prompt, model, and API modules. The **API** represents the underlying application programming interface exposed by the concrete LLM. With this, HoneyLLMd can use the API based URL. In our case, the API is only required to provide an OpenAI-compatible chat completion endpoint. The **Prompt** and **Model** stand behind the the API. Prompt provides contextual constraints, role specification, and response control, which collectively guide the LLM to generate outputs that are coherent with the task requirements. Our design is not limited to a certain large language model, while we even lean toward to the plug-in architecture for the sake of extendable implementation.

D. Honeypot Modeling

1) **Data Capture Capability Vs. Risk of Compromise**: As we know that there is always a trade-off between the data capture capability and the risk of compromise in honeypots. Thus, modeling the trade-off between the risk of compromise (security level) and data capture capability (ability to attract malicious activities) in a honeypot is an interesting and arduous challenge. The honeypot design objective is to maximize attack data capture capability while minimizing the security risk.

We define $x \in [0, 1]$ as the exposure level, i.e., the amount of the exposed information system resources (e.g., vulnerable services), where '0' indicates minimal exposure and '1' denotes full exposure; thus, $\lambda(x)$ is the data capture capability, which is a function of x ; and also, $p(x)$ is the probability of honeypot being fully compromised, whereby $p(x)$ must be constrained to an acceptable risk threshold. Therefore, the honeypot objective function is to maximize the following with respect to x , i.e., controlling x :

$$\max_{x \in [0, 1]} \lambda(x) - \alpha \cdot p(x), \quad (1)$$

where α is a risk-aversion parameter.

Taking advantage of using LLM to emulate the real system response for the honeypot, x in (1) can be optimized to '1', however, the whole honeypot system will never be compromised because there is no real system behind, which is the beauty of the LLM-powered honeypot.

2) **Hierarchical Probabilistic Automaton**: An HPA is a powerful computational model, which is essentially a state machine where states can themselves be other probabilistic automata. This allows HPA to model complex, multi-stage processes in a manageable and compact way, significantly enhancing the ability to estimate attack vectors and dynamically adapt engagement strategies.

As shown in Fig. 3, the commands will be divided into sub-commands if possible, and each of them can be independently treated as an automaton. Within each automaton, the structure is first captured as process trees, which describe the parent-child relationships among states, and the primary program invoked is identified as the macro state, while the arguments associated with that program are represented as its micro states. From these process trees, process vectors are derived, describing the ordered sequence of programs executed during an attack. All process

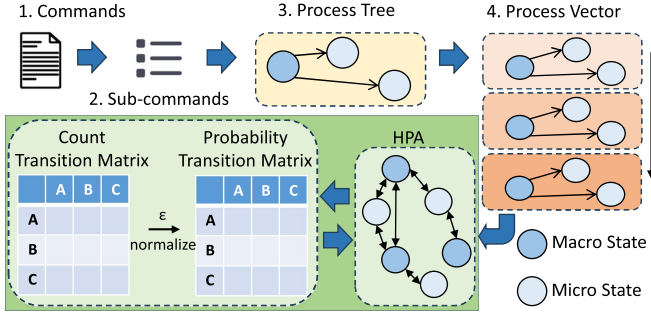


Fig. 3. Process of HPA Modeling.

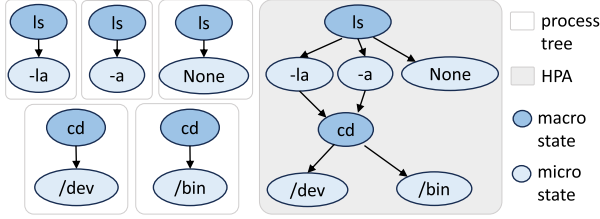


Fig. 4. Example of HPA Generation.

vectors observed from attacks are then aggregated into an overall HPA, which encodes the global transitions among states. From this representation, two $N \times N$ square matrices are derived, namely probability transition matrix M and count transition matrix C . In these two matrices, both the first row and the first column correspond to the complete set of HPA states. In matrix C , each entry C_{ij} records the count of transitions from state i to state j , thereby capturing the distribution of attacker behaviors. To transform these frequencies into matrix M , the outgoing transitions of each state are normalized such that their cumulative probability sums to one. Furthermore, a small smoothing factor ε is incorporated during this process to prevent zero-probability cases. Fig. 4 illustrates the HPA generation procedure. In this example, an attack session consists of a sequence of three commands: “ls -la | cd /bin”, “ls -a | cd /dev”, and “ls”. These commands are decomposed into five distinct automata, each defined by its specific macro and micro states. For instance, the command “ls -la | cd /bin” is parsed into two sub-commands: the first automaton is represented by the macro state “ls” and the micro state “-la”. Therefore, the session is transformed into the process vector: $\langle \text{ls}, -\text{la}, \text{cd}, / \text{bin}, \text{ls}, -\text{a}, \text{cd}, / \text{dev}, \text{ls} \rangle$, and the HPA is generated accordingly.

3) *Strategy of Adaptation*: Built upon the HPA modeled attack process, we are able to adapt the deception defense strategy, whereby the honeypot payoff is stressed on in order to optimize the honeypot benefit but still engage the attack activity. To quantitatively evaluate the interaction between the attacker and the honeypot, payoff captures how likely the attacker’s chosen path is compared to the most probable path. Formulas (2) and (3) define the attacker’s payoff R_a^p and honeypot’s R_h^p respectively:

$$R_a^p = \frac{P_{act}}{P_{opt}} \quad (2)$$

$$R_h^p = 1 - \frac{P_{act}}{P_{opt}}, \quad (3)$$

where P_{act} denotes the probability of the transition actually chosen by the attacker, and P_{opt} denotes the probability of the most probable (optimal) path computed from the matrix M . When the attacker follows the most probable path, the attacker’s payoff approaches 1 while the honeypot’s payoff converges to 0. In contrast, when the attacker deviates due to honeypot interference, the honeypot’s payoff is maximized. Specifically, when a new state transition that does not exist in the matrix is observed, the P_{act} and P_{opt} are recognized as *None* since the transition probability is absent from the underlying data. In this case, the attacker’s payoff R_a^p is assigned a value of with 0, which indicates that the attacker has not gained any effective progress toward the objective, and the honeypot’s payoff R_h^p is valued as 1, which means the honeypot has recorded a new attack path.

Dynamic update of the state transition matrix: Since attacker behavior evolves continuously during interaction, the matrix C and matrix M must be dynamically updated to reflect the most recent observations. Incoming inputs are processed in the same manner as in HPA construction (see Section III-D2), where each input is mapped into a corresponding state representation.

The system would update the count transition matrix C by incrementing the frequency if the transition already exists, or by adding a new entry if it is observed for the first time. Next, the probability transition matrix M is derived. In practice, there are three distinct updating cases of as (4) shows:

$$P_{ij} = \begin{cases} \frac{C_{ij}}{\sum_j C_{ij}} \cdot (1 - \varepsilon), & \text{if } C_{ij} > 0, \\ \frac{\varepsilon}{N - n}, & \text{if } C_{ij} = 0 \text{ and } \sum_j C_{ij} > 0, \\ \varepsilon, & \text{if } \sum_j C_{ij} = 0 \end{cases} \quad (4)$$

- 1) For observed transitions ($C_{ij} > 0$), the probability is computed by multiplying the normalized transition frequency by $(1 - \varepsilon)$.
- 2) For unobserved transitions from active states ($C_{ij} = 0$ and $\sum_j C_{ij} > 0$), the probability is set to $\varepsilon/(N - n)$, distributing the smoothing factor evenly among all $N - n$ missing transitions, where n is the number of observed transitions from state i .
- 3) For completely new states with no recorded transitions ($\sum_j C_{ij} = 0$), there is no prior information to estimate the likelihood of any particular transition.

A probability of $1/N$ is assigned to the smoothing factor ε , ensuring that the system remains unbiased and capable of adapting as new observations are collected, while keeping the matrix M valid for subsequent updates.

Algorithm for path searching: To efficiently identify the the actual transition probability P_{act} and optimal path probability P_{opt} , we reformulate the problem as a shortest-path search on a weighted graph. As Fig. 5 shows, each transition $p = M_{ij}$ in the probability matrix is transformed into an edge weight. So that the task of maximizing path probability is equivalent to minimizing cumulative weight. Specifically, a higher probability yields lower edge weight. This transformation enables Dijkstra’s algorithm to explores states by increasing path cost, ensuring the optimal path is found.

Algorithm 1: Dynamic Optimal Path Search with Probability Threshold.

Input: Transition probability matrix M ; start state s ; end state t ; minimum probability threshold $p_{\min}$
Output: Actual probability P_{act} ; optimal probability P_{opt}

Initialization;

$w(u, v) \leftarrow -\log M[u, v]$ for all $M[u, v] > 0$;
 $\text{dist}[u] \leftarrow \infty, \forall u$;
 $\text{dist}[s] \leftarrow 0$;
priority queue $Q \leftarrow \{(0, s)\}$;
 $\text{max_cost} \leftarrow -\log(p_{\min})$;

while Q not empty **do**

($\text{cur_dist}, u$) $\leftarrow \text{pop}(Q)$;
if $\text{cur_dist} > \text{dist}[u]$ **then**
 continue;
if $\text{cur_dist} > \text{max_cost}$ **then**
 break;
if $u = t$ **then**
 $P_{\text{opt}} \leftarrow e^{-\text{cur_dist}}$;
 break;
foreach v such that $M[u, v] > 0$ **do**
 $\text{new_dist} \leftarrow \text{cur_dist} + w(u, v)$;
 if $\text{new_dist} < \text{dist}[v]$ **then**
 $\text{dist}[v] \leftarrow \text{new_dist}$;
 push ($\text{new_dist}, v$) into Q ;

$P_{\text{act}} \leftarrow M[s, t]$;
return $P_{\text{act}}, P_{\text{opt}}$;

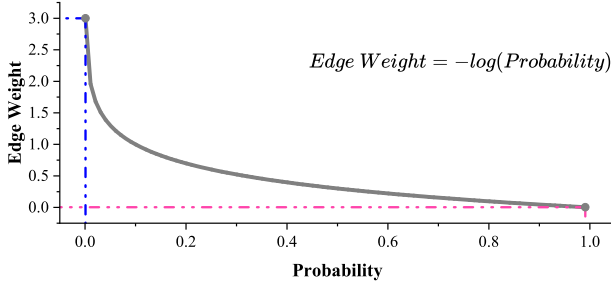


Fig. 5. Relationship between Probability and Edge Weight.

To ensure scalability on large $N \times N$ matrices, a probability threshold $p_{\min}$ is introduced to prune infeasible paths. A path $\pi = (i_1, i_2, \dots, i_k)$ is retained only if its cumulative probability satisfies

$$P(\pi) = \prod_{r=1}^{k-1} M_{i_r, i_{r+1}} \geq p_{\min} \quad (5)$$

We can generally select a fixed minimum cumulative probability threshold $p_{\min}$ to prune infeasible paths without dynamically computing it for each update. Considering the size of the state space ($N \geq 8000$) and the typical path lengths in our system, a value of $p_{\min} = 10^{-9}$ is chosen. This threshold is sufficiently small to retain realistic paths while efficiently discarding sequences with extremely low probability, thereby reducing the search space and improving computational efficiency.

Algorithm 1 presents the detailed procedure, which takes the probability transition matrix M , start and end states (s, t), and the threshold $p_{\min}$ as input, and returns the values of P_{act} , P_{opt} . The complexity of Algorithm 1 involves two main aspects,

i.e., the initialization of the weight matrix and the shortest-path search with probability threshold. First, the initialization of the weight matrix requires computing $-\log M_{ij}$ for each non-zero transition probability, which is linearly related to the number of edges E , namely, $O(E)$. Second, the shortest-path search is similar to Dijkstra's algorithm and involves updating distances and priority queue operations for each node and edge. Using a heap-based priority queue, the complexity of this search is $O((V + E) \log V)$, where V denotes the number of states. The pruning procedure does not change the worst-case complexity. Therefore, the overall time complexity of the algorithm is $O((V + E) \log V)$. Regarding space, the algorithm stores the weight matrix, distance array, and priority queue, leading to a space complexity of $O(V + E)$.

Block mechanism and corresponding LLM responses: After modeling the attacker's path and training the HPA on historical attack chains, we compute the attacker's payoff (R_a^p). This value represents the current probabilistic intent of the attacker to reach the target malicious state (e.g., payload execution), based on the observed HPA state transitions. The honeypot implements a security policy governed by a Block Threshold $B \in [0, 1]$. The decision is made dynamically: when the calculated attacker payoff (R_a^p) exceeds this threshold ($R_a^p > B$), the system adaptively enforces a block, forcing the attacker to diverge from their most optimal path. Given the complementary relationship between the honeypot and the attacker's payoff ($R_a^p + R_h^p = 1$), we aim to ensure that a block threshold is found such that the payoff of the honeypot is at least greater than 0.5.

When the attacker is blocked, HoneyLLMd calls the LLM API to generate deceptive responses, such as "permission denied" or "timeout", so that the attacker perceives the command as unsuccessful. In contrast, if the input is allowed, HoneyLLMd generates valid simulation responses, making the attacker believe that the command has been successfully executed. By adaptively adjusting this blocking policy, the HoneyLLMd is able to enforce fine-grained control while maintaining a coherent and adaptive strategy for engaging the attacker, ultimately enabling richer observation and capture of attacker activities.

E. Deception Evaluation Metrics

We define the deception evaluation metrics for honeypots from two perspectives:

- 1) *The system's ability to successfully execute and respond to an attacker's commands:* When an attacker initiates an attack, a honeypot, as a protective system, should provide the responses that the attacker expects to lure them into further attacks, thereby collecting more information about the attacker. Therefore, successfully answering the attacker's commands and providing the information they seek is crucial.
- 2) *The system's responses should align with the logic of the OS:* When the system responds to the attacker's commands, these responses should conform to the logic of the current OS to avoid being detected by the attacker. In addition, when the attacker sends probing commands that

TABLE II
DECEPTION PERFORMANCE CATEGORIES

Deception Categories	Successful Attack	Logic Compliance
Successful Attack and Logic Compliance (SALC)	✓	✓
Successful Attack but no Logic Compliance (SANLC)	✓	✗
Failure Attack and Logic Compliance (FALC)	✗	✓
Failure Attack but no Logic Compliance (FANLC)	✗	✗

do not follow logical patterns, the honeypot should still be able to provide a response that aligns with what the OS would do, thereby evading the attacker’s probes.

Based on these two aspects, we propose four categories as in Table II to calculate the metrics of deception performance. We use real system responses as the oracle of the four categories. The “successful attack” item indicates the system successfully executes the command and provides effective response information when the attacker issues a command. The “logic compliance” item holds when the system response adheres to the logic of a real OS.

Based on these four categories, we define four metrics to evaluate the deception performance of honeypots:

- *Precision*: This represents the precision of the system in successfully executing attack commands. A higher value indicates that the system closely mimics a real system when successfully executing commands.

$$\text{Precision} = \frac{\text{SALC}}{\text{SALC} + \text{SANLC}} \quad (6)$$

- *Temptation*: This metric represents the system’s attractiveness to attackers, which is the proportion of successfully executed commands among all responses that comply with the OS’s logic. A higher value means that the system provides more valuable information to attackers, making it more realistic and attractive.

$$\text{Temptation} = \frac{\text{SALC}}{\text{SALC} + \text{FALC}} \quad (7)$$

- *Attack Success Rate*: The proportion of successful attack commands among the total number of commands. It indicates the proportion of commands that attackers can successfully execute. A higher value suggests that the system often allows attackers to succeed in their attacks, thereby providing more information to them.

$$\text{Attack Success Rate} = \frac{\text{SALC} + \text{SANLC}}{\text{SALC} + \text{SANLC} + \text{FALC} + \text{FANLC}} \quad (8)$$

- *Accuracy*: It is a crucial metric that signifies the system’s resemblance to a real system. A higher accuracy indicates that the system closely mimics a real system, thereby effectively deceiving attackers.

$$\text{Accuracy} = \frac{\text{SALC} + \text{FALC}}{\text{SALC} + \text{SANLC} + \text{FALC} + \text{FANLC}} \quad (9)$$

IV. IMPLEMENTATION

This section presents the implementation details of the proposed approach.

A. Prototype Implementation

We developed a modular prototype of HoneyLLMd using programming language Python 2.7. The system leverages the `asynchat` framework for asynchronous I/O, allowing it to manage multiple concurrent attacker connections. Furthermore, we integrated OpenAI APIs via OpenRouter.¹ These APIs are invoked by specifying the model identifier and their corresponding endpoint. The model prompts are defined within the system configuration, and a cache recording the session history in JSON format was utilized to reduce the API latency. The transition matrices are stored as CSV files that can be dynamically updated during the interaction. The block mechanism is enabled or disabled via a boolean flag defined in the configuration file. When we disable the block mechanism, it is equivalent to the block threshold that equals 0. The shell decoder was realized using two approaches, namely the RE-based decoding and LLM-based decoding. The implementation for the shell decoder is detailed in Section IV-B.

B. Shell Decoder

We generated the reverse shell obfuscated payloads that operate across eight common script languages-including Bash, Perl, Python, PHP, Ruby, Netcat, PowerShell, and Lua, and eight common obfuscation and encoding techniques under a controlled testbed environment. We also augmented the dataset with normal payloads which contains benign samples. For each of the eight languages we added a set of harmless command snippets and benign code examples (e.g., file listing, simple I/O, language-specific no-op snippets and benign network requests). In addition, we included a subset of benign strings called hard negatives that intentionally contain superficially suspicious artefacts. These resulting malicious payloads were only used for recognition under both methods in a controlled test environment and did not involve actual execution. The obfuscation techniques are detailed as follows:

- *Base64 Encoding*: A method of encoding binary data into a text representation using the Base64 encoding scheme. Commonly used to obfuscate data or embed it in text-based formats.
- *URL Encoding*: A technique used to encode special characters in a URL to ensure that they are transmitted correctly. Characters are replaced with a ‘%’ followed by their hexadecimal ASCII code.
- *Hexadecimal Encoding*: A method of encoding binary data into a text representation using the hexadecimal format. Often used to represent binary data in a human-readable form.

¹<https://openrouter.ai/>

- *String Concatenation*: The process of joining multiple strings together to form a single string. This technique is usually used to dynamically construct payloads or commands.
- *Inline Comments*: Comments inserted directly into code to obscure the actual logic or intent of the code. This technique complicates the static analysis and reverse engineering process.
- *Environment Variable Injection*: A technique where parts of the payload or command are stored in environment variables and then combined or executed dynamically at runtime.
- *Environment Variable Concatenation*: The technique of splitting a payload into multiple environment variables and then concatenating them at runtime to reconstruct the original payload.
- *Multi-layer Encoding*: The use of multiple encoding techniques (e.g., Base64 combined with string concatenation) to further obscure the payload and make detection and analysis more challenging.

C. Dataset Statistics and Preprocessing

We employed two real-world open-source datasets collected by Cowrie honeypots deployed in cloud-based environments. Both datasets consist of SSH and Telnet interaction logs, with the honeypots listening on ports 22 and 23, respectively. The dataset 1, collected by [91], was replayed to the system to evaluate the LLM capability in terms of shell decoding and deception performance, and was used to construct the HPA transition matrices. It contains 214,132 log entries during 9 June to 4 July 2022, consisting of 82 unique commands. Dataset 2 [92] is a publicly available dataset from Kaggle. It contains 42,817 log entries and 67 unique commands during the period of October 2024. It is under a CC BY-SA 4.0 license and is used exclusively as the attack&evaluation dataset against the HPA-enabled honeypot. These datasets are kept separately to prevent training/evaluation contamination.

We categorized and compared the attack categories of these datasets by classifying them at the session level using the MITRE ATT&CK² Enterprise Matrix for Linux platform. This framework reflects the current knowledge base of adversary tactics and technologies that has been formed based on actual observations. Each tactic and technique has been described in detail and has been assigned a unique identifier. Table III lists the tactics reflected in the datasets and the corresponding technique types. Compared with the official techniques, both of these datasets cover most of the threat types. However, they differ in the emphasis of the attack categories. Specifically, dataset 1 shows a higher concentration in activities such as Execution, Persistence, Privilege Escalation, and Command and Control operations. In contrast, dataset 2 focuses more on Discovery techniques and includes 19 of them. It also covers most of the techniques of Initial Accesses. As the datasets are initially raw logs, a systematic preprocessing procedure for datasets

TABLE III
ATTACK CATEGORIES DISTRIBUTION UNDER MITRE ATT&CK ENTERPRISE MATRIX (LINUX)

Tactic ID	Tactic Name	Official Tech.	Dataset 1 Tech.	Dataset 2 Tech.
TA0001	Initial Access	10	3	6
TA0002	Execution	10	7	3
TA0003	Persistence	18	14	1
TA0004	Privilege Escalation	12	7	2
TA0006	Credential Access	15	3	1
TA0007	Discovery	26	2	19
TA0011	Command and Control	18	8	3

Note: This table only shows the matrices reflected in the databases.

was applied. First, all input-related events were extracted from the raw JSON logs, while retaining essential metadata such as session identifier, source address, and timestamp. Second, events were grouped into sessions according to their identifiers, and within each session the commands were ordered based on their timestamps to preserve the natural execution sequence. Third, compound commands (e.g., involving pipes, conditional operators, or semicolons) were further decomposed into sub-commands using regular-expression-based parsing rules, while ensuring that quoted strings and protected segments were not erroneously split.

D. Deception Metrics Labeling Procedure

To implement the deception evaluation metrics, we developed an automated labeling pipeline using Python and LLM. The procedure uses the real system output (R_{ora}) as the oracle to categorize the LLM response (R_{llm}).

The first layer of the pipeline determines the “successful attack” status by scanning for execution failure indicators. We utilize RE to identify a predefined and continuously updated set of failure keywords, such as `error`, `EOF`, `not found`, and `no/not such file`. If R_{llm} does not contain these failure indicators, it is initially categorized as either SALC or SANLC. Otherwise, it is categorized as either FALC or FANLC. Also, the LLM would be used to divide the response into logical compliance and logical non-compliance. If the response adheres to one of the following four conditions, it would be classified as logical non-compliance:

- *Syntactic Incorrectness*: If R_{llm} is not in the shell format of the currently specified system, but rather a response in other format (e.g., nature language).
- *State Inconsistency*: If R_{llm} reflects a system state that contradicts prior interactions. For instance, if an attacker successfully deletes a directory, but R_{llm} still lists it in subsequent commands.
- *Environment Nonalignment*: If R_{llm} references tools or paths inconsistent with the target OS (e.g., a `yum` error on an Ubuntu target).
- *Permission Non-compliance*: If R_{llm} indicates an opposite permission result with R_{ora} .

E. Testbed Deployment

To evaluate the capabilities of HoneyLLMd against traditional MIHs, such as Amun [15] and Cowrie [17], they were deployed

²<https://attack.mitre.org/>

TABLE IV
STATISTIC COMPARISON OF VARIOUS LLMs

Model	Version	Context	Cost (USD / 1M Tokens)		
			Cached Input	Input	Output
Gemini	2.5-pro	976k	–	1.25	10.00
	2.5-flash	976k	–	0.3	2.5
GPT	5	–	0.125	1.25	10.00
	5-mini	–	0.025	0.25	2.00
	4o	–	1.25	2.50	10.00
	4o-mini	–	0.075	0.15	0.60
Claude	Sonnet-4	–	0.30	3.00	15.00
	Opus-4.1	–	1.50	15.00	75.00
	Haiku-3.5	–	0.08	0.80	4.00

Note: Pricing data are collected from OpenRouter on Sept. 23, 2025. Values may change over time. “–” indicates unavailable information.

on CloudLab.³ Each honeypot was hosted on a dedicated VM configured with 4 vCPU cores and 4096 MB of RAM running Ubuntu 22.04. The VMs were configured with public accessible IP addresses for collecting and comparing real-world attack data.

To validate the effectiveness of HoneyLLMd against human-driven attack scenario, we conducted a controlled cyber-range simulation. We employed the CTFAgent [93], an autonomous capture the flag (CTF) framework, as the offensive agent to emulate malicious activities. It is supported by an LLM-driven engine with Chain-of-Thought (CoT) reasoning to adapt its attack strategy based on feedback. The simulation was conducted within a controlled environment, utilizing two VMs. Both VMs share an identical configuration, running Ubuntu 22.04 with 4096 MB of RAM. To facilitate communication, the VMs are configured with bridged adapters in promiscuous mode, placing them within the same subnet. This isolated setup ensures that traffic is restricted exclusively to these two nodes, preventing any interference with external network infrastructure.

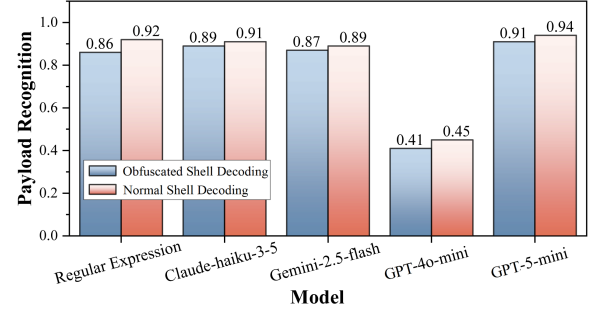
V. EXPERIMENTS

This section presents the experimental results obtained from the prototype implementation along with the corresponding analysis and discussion.

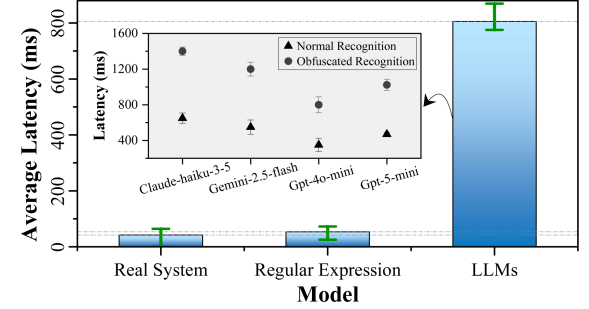
A. Comparison of Large Language Models

As the LLM plays an essential role in our design as shell decoder and session responder, Table IV compares four official indexes^{4 5 6} of a number of common LLMs. The context index refers to the maximum number of tokens that the model’s word interactions can handle and remember. For example, Gemini-2.5-pro can memory information related to 976,000 tokens. The input, cached input, and output indexes explain the cost of invoking the API of corresponding model.

Context and cached input cost determine the scope of the context that our system can handle. We hope that Context can be as large as possible or the cached input cost can be as small as possible, so as to obtain the largest historical record space at



(a) Payload Recognition Correctness.



(b) Payload Recognition Average Latency.

Fig. 6. Payload Recognition Capability Comparison.

the minimum cost. Similarly, the input and output cost should also be controlled. Therefore, GPT-4o and Claude-Opus-4.1 are not preferred since they cost relatively high on catching. Moreover, GPT-5, Claude-Sonnet-4, and Gemini-2.5-pro are not considered since their output cost are expensive for invoking. As a result, we evaluate the rest of models in the following experiments.

B. Comparison of Shell Decoder Performance

As aforementioned, the selected testing LLMs include GPT-5, GPT-4o-mini, Gemini-2.5-flash, and Claude-haiku-3.5. To assess the performance of the LLM-based shell decoder, we test the payload recognition correctness (Fig. 6(a)) and the payload recognition latency (Fig. 6(b)). We define payload recognition correctness as the ability of the analyzer agent to accurately identify and extract payload information, and payload recognition latency as the interval to finish the recognition procedure. Using this definition, we employ 91 payloads (64 obfuscated payloads and 27 normal payloads) as benchmarks to evaluate the five analyzer agents.

Fig. 6(a) shows that the correctness of normal payload is generally higher than the obfuscated one. Specifically, regular expression exhibits a dissatisfied result on obfuscated payload recognition correctness as their are 12 payloads are not correctly recognized, where 4 benign payloads are mismatched and 8 obfuscated payloads are missed. GPT-5-mini, Claude-haiku-3.5, and Gemini-2.5-flash demonstrate excellent performance, with obfuscated payload recognition accuracy rates generally above 90%. In contrast, GPT-4o-mini shows a marked decrease in recognition correctness for both normal and obfuscated payloads. In addition, Fig. 6(b) shows the average latency of payload

³<https://www.cloudlab.us/>

⁴<https://openai.com/api/pricing>

⁵<https://www.anthropic.com/api#pricing>

⁶<https://deepinfra.com/pricing>

recognition under different instances. RE's latency (53.48 ms) delivers a ideal value that is similar to the real system (41.95 ms) within a smaller error range. However, LLMs exhibit a worse performance compared to the real system and RE with an average latency of 805.38 ms. The detailed sub figure reveals the latency of four LLMs, and it is clear that the latency of obfuscated recognizing is usually longer than normal one. Although GPT-4o-mini exhibits the smallest latency in both two recognition tasks, its recognition accuracy is much worse than other models, indicating it is not a good choice for payload recognition tasks.

Overall, RE seems to be an appropriate choice for shell decoding. However, RE's limitations stem from its inherently rigid nature, which can struggle to adapt to the diverse and complex patterns present in different shellcode types. In contrast, leveraging the capabilities of LLMs has shown a promising ability to enhance the effectiveness of shellcode decoding. The advanced pattern recognition and learning abilities of LLMs enable them to better understand and interpret the complexities of obfuscated code, offering a more versatile solution to the challenge of shellcode decoding.

Therefore, while RE-based decoding aligns more closely with expected service latency and offers a straightforward approach, incorporating LLMs into the decoding process can significantly enrich the toolset available for tackling sophisticated obfuscation techniques in shellcode, albeit with a trade-off in terms of increased latency. This suggests a potential area for further research and development, aiming to optimize the balance between decoding accuracy and efficiency when dealing with complex cybersecurity threats.

Insight 1: Advanced LLMs demonstrate remarkable correctness in recognizing and analyzing the shell code, even when the shell code is obfuscated. However, its effectiveness with zero-day exploitations has yet to be determined.

C. Comparison of Deception Performance

We evaluate the deceptive capabilities of various LLMs with no block threshold to facilitate model selection. We identified the dataset [91] to assess six different systems across four predefined categories, using a real system and a Cowrie honeypot as baselines for comparison. We use SSH to connect to each of them, and perform traffic replay to emulate the real attack scenario.

From Fig. 7, it can be observed that in the test of 82 distinct commands, the real system was able to successfully respond to 28 commands from the attacker, while the remaining 54 commands could not provide the desired information due to permission restrictions or other security mechanisms. The LLM-powered HoneyLLMs show a higher average SALC value than real system and Cowrie, indicating LLMs provide more executable and reasonable response. However, Cowrie's performance was less than satisfactory, which only provided 24 SALC items, and 25 FALC items. The remaining 33 responses fell into the categories of logically inconsistent responses (SANLC and FANLC). The result indicates that Cowrie's simulation of the OS was not realistic enough to conceal the identity of a honeypot, and not deceptive enough compared with an LLM.

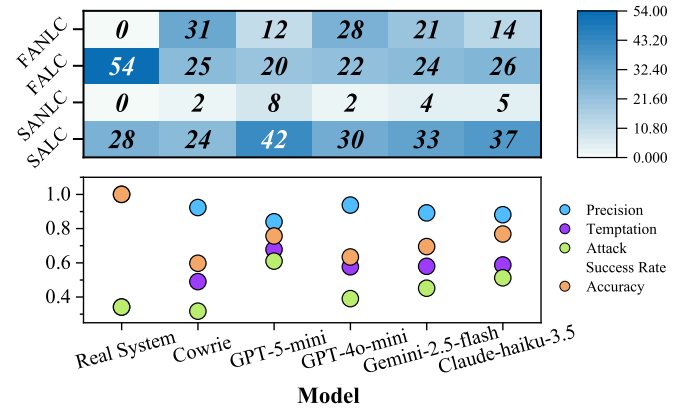


Fig. 7. Deception Performance Comparison between Real System and Proposed HoneyLLMs.

As the Fig. 7 shows, apart from GPT-5-mini, Cowrie and the other LLMs perform similarly in terms of precision, with values approaching 90%. Moreover, LLM-powered HoneyLLMs outperform Cowrie in terms of temptation, attacker success rate, and accuracy. It is clear that four LLMs are able to provide a higher temptation value. In particular, GPT-5-mini exhibits the most significant improvement in temptation, surpassing Cowrie by approximately 68%. This indicates that, compared to Cowrie, it is able to execute more commands and provide responses that appear substantially more attractive to attackers. Meanwhile, GPT-4o-mini, Gemini-2.5-flash, and Claude-haiku-3.5 all achieve temptation levels around 58%. Although not as high as GPT-5-mini, these results still demonstrate stronger engagement capabilities than Cowrie. With respect to the attack success rate, LLM-powered models generally achieve higher values than the real system.

Among them, GPT-5-mini reaches 60%, highlighting its strong capacity to consistently generate outputs that mimic command execution. This suggests that GPT-5-mini is especially effective at maintaining the illusion of successful execution, thereby sustaining the attacker's engagement. Claude-haiku-3.5 follows with a score of 51%, marking it as the second-best performer. In contrast, GPT-4o-mini shows a weaker ability in this aspect, reaching only 43%, which is merely 4% higher than Cowrie. Finally, regarding accuracy, Claude-haiku-3.5 achieves the best performance with 76%, followed closely by GPT-5-mini with 75%. These results indicate that both models are highly capable of generating system responses that align with realistic behaviors, thereby enhancing the credibility of the deception. Overall, the results demonstrate that LLM-powered honeypots surpass Cowrie in both authenticity and persuasiveness of responses. Among the tested models, GPT-5-mini and Claude-haiku-3.5 emerge as the strongest candidates for integration into HoneyLLM, as they combine strong engagement potential with a high degree of response fidelity.

D. Dataset Similarity

As the datasets is the basic of HPA and affects a lot on the payoff and block results. We first analyze the similarity and

TABLE V
COMPARISON OF CAPTURED TOP 10 COMMANDS BETWEEN DATASETS

ID	Top 10 Commands in Dataset 1	Count	Top 10 Commands in Dataset 2	Count
1	cat /proc/cpuinfo grep name head -n 1 awk	4327	uname -s -v -n -r -m	959
2	free -m grep Mem awk '{print \$2, \$3, \$4, \$5, \$6, \$7}'	4325	sh	269
3	ls -lh \$(which ls)	4323	enable	268
4	which ls	4323	ping; sh	259
5	crontab -l	4322	/bin/busybox cat /proc/self/exe cat /proc/self/exe	259
6	w	4322	empty	259
7	cat /proc/cpuinfo grep name wc -l	4315	cd && rm -rf .ssh && mkdir .ssh && echo "ssh-rsa AAAAB3Nza..." » .ssh/authorized_keys && chmod -R go=7.ssh && cd	220
8	uname -m	4314	cd ; chattr -ia .ssh; lockr -ia .ssh	220
9	cat /proc/cpuinfo grep model grep name wc -l	4284	cat /etc/*release* grep 'centos:7'	38
10	top	4249	uname -a	25

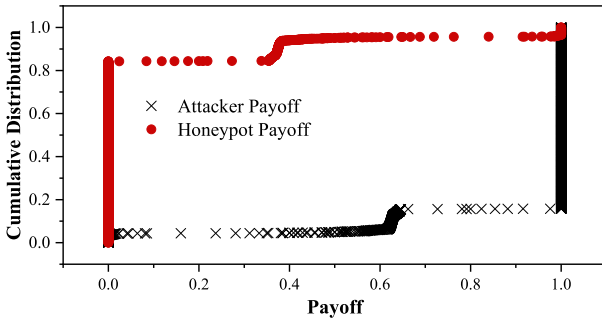


Fig. 8. Payoff Distribution.

commands distribution of the two datasets.

$$\mathbf{v}_{\text{set}} = \frac{1}{N_{\text{set}}} \sum_{i=1}^{N_{\text{set}}} \mathbf{x}_i \quad (10)$$

$$\text{Cosine Similarity} = \frac{\mathbf{v}_{\text{set 1}} \cdot \mathbf{v}_{\text{set 2}}}{\|\mathbf{v}_{\text{set 1}}\| \|\mathbf{v}_{\text{set 2}}\|} \quad (11)$$

To assess the similarity of these two datasets, we calculate the average vector of them with Formula (10), and the cosine similarity of the command distributions according to Formula (11) is only approximately 0.15, indicating that the attacker behaviors in 2022 and 2024 have undergone a substantial shift. Table V presents the top-10 most frequent commands of two datasets respectively. The results reveal a remarkable divergence: commands in the set [91] are dominated by information-gathering and system profiling activities (e.g., querying CPU, memory, and system binaries), while those in set [92] primarily consist of persistence, reverse shell, and BusyBox-related behaviors (e.g., injecting SSH keys, executing sh, and issuing chained commands).

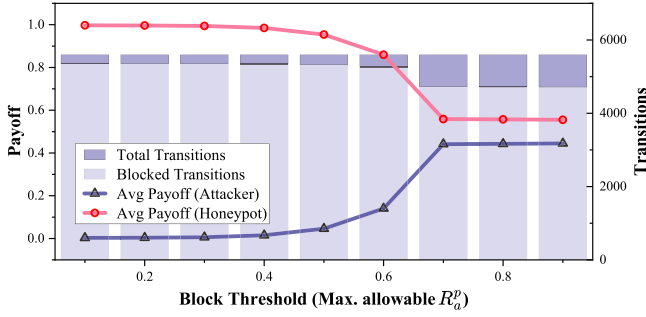
E. Payoff Analysis and Block Threshold Evaluation

1) *Payoff Distribution*: We depict the payoff distribution between the attacker and the honeypot in Fig. 8. It is clear that over 80% of attacker's payoffs clustering near 1.0, whereas the honeypot's payoffs remain predominantly near 0.0. Owing to the definition of payoff, the payoff distributions of

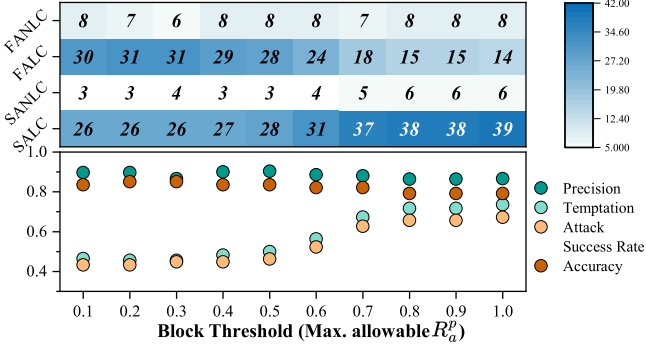
the attacker and the honeypot are inherently mirror images of each other. The disparity is reasonable given the low similarity between the datasets [92] and [91]. HoneyLLMd tends to assign the new transitions with high payoff during each update and therefore block them. By giving such transitions high payoff, the system deliberately encourages the attacker to execute more commands. This behavior shows that the real-time decision logic of our system is effective in steering attacker actions and improving capture of malicious activities.

2) *Sensitivity Analysis of Block Threshold*: To determine the appropriate block threshold, achieving the trade-off between data capture capability and system risk, we evaluate the average payoff of the attacker and the honeypot, the number of blocked transitions, and the deception performance under different block thresholds. Based on the previous experimental results, we adopt GPT-5-mini for this evaluation.

As shown in Fig. 9(a), of unblocked transitions occurs when the block threshold increases from 0.1 to 0.6: the majority of transitions remain blocked (95%), and the attacker's average payoff increases slowly. Specifically, when the threshold crosses 0.6, the attacker's average payoff jumps markedly (from 0.14 to 0.44) while the honeypot's payoff and the blocked-transition ratio both drop substantially. This sharp transition indicates a loss of the honeypot's protective margin: permitting a higher threshold yields little additional capture benefit but substantially increases compromise risk. As Fig. 9(b) depicted, when the block threshold is within the range of 0.1 to 0.6, the number of "successful attack" (SALC and SANLC) slightly increases. However, when the threshold exceeds 0.6, there is an evident raise of SALC (from 31 to 37) and reduction of FALC. Accordingly, the Temptation and Attack Success Rate ascend to approximately 0.7, indicating the improve of system's attract ability, but also suggesting a higher risk of the system being breached. Overall, the number of SANLC and FANLC keeps stable. Precision and Accuracy remain robust at approximately 0.82. Although both metrics exhibited a marginal decline as the threshold increased, the magnitude of this decrease was negligible. This suggests that HoneyLLMd can exhibit consistent emulation fidelity relative to the real OS across varying thresholds.



(a) Payoff Variation under Different Block Thresholds.



(b) Deception Performance under Different Block Thresholds.

Fig. 9. Block Threshold Evaluation.

TABLE VI
PERFORMANCE COMPARISON OF HONEYLLMD

Performance Indicator	HoneyLLMd (GPT-5-mini)		Variance
	LLM-only ($B = 1$)	LLM+HPA ($B = 0.6$)	
Precision	0.867	0.886	+0.019
Temptation	0.736	0.563	-0.173
Attack Success Rate	0.672	0.522	-0.150
Accuracy	0.791	0.820	+0.029
Session Duration	Max.	242.53s	390.67s
	Avg.	63.64s	76.41s
Collected Unique Commands	1505	2842	+1337

Therefore, we identify a threshold of 0.6 as the operationally optimal trade-off where honeypot system is able to retain relatively high data capture capability while keeping the risk of system being compromised at an acceptable and relatively low level.

F. Evaluation of HoneyLLMd

1) *Ablation Study*: To clarify the contributions of LLM and HPA in the proposed system, we compare the deception performance, session duration, and the unique commands collected by HoneyLLMd under two configurations: the ‘LLM-only’ version sets the block threshold $B = 1$, rendering the HPA mechanism disabled; the ‘LLM+HPA’ version utilizes the evaluated threshold of $B = 0.6$ (the optimal value in terms of Section V-E2 maintaining relatively high honeypot payoff and acceptable system compromise risk), representing the fully-functioned honeypot system. The upper half of Table VI details the data of deception

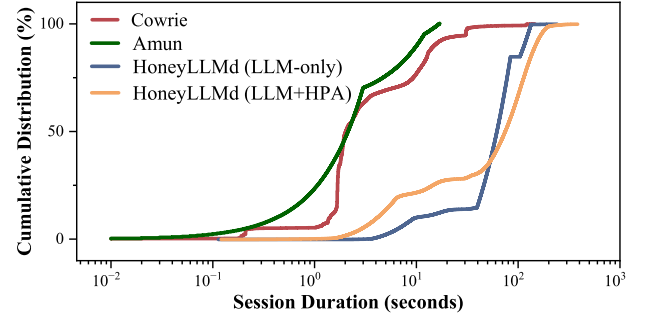


Fig. 10. Session Duration Comparison.

metrics. The rest shows the session data collected after cloud deployment.

Firstly, the integration of HPA enhances the system’s Precision (+0.019) and Accuracy (+0.029). This improvement suggests that HPA is conducive to emulating the OS. In addition, a reduction in Temptation (−0.173) and Attack Success Rate (−0.15) are also observed. A lower Attack Success Rate indicates that the HPA module successfully intercepts high-payoff attacker commands that could potentially compromise the honeybot’s true identity. Secondly, Fig. 10

further plots CDF curves of connection duration across Cowrie, Amun, and HoneyLLMd. Specifically, Amun shows the worst performance on session duration: most of the connections can merely last for 5 to 10 seconds. Since the Amun honeypot only hard-codes little response logic, it is unable to cover all possible response scenarios for attack commands, nor can it dynamically change the response strategy during the interaction. Cowrie performs better than Amun, as evidenced by the longest duration of Cowrie reaching nearly 100 seconds, and almost all of the connection durations were longer than those of Amun. However, it still struggles to sustain extended conversations due to its hard-coded nature. Notably, as LLMs are able to dynamically generate responses, they demonstrate superior duration performance. They are capable to maintain at least 50% of the interactions for 20 s, and at the same time, the maximum duration can reach the hundreds of seconds level. Benefit from the adaptive HPA mechanism, fully-functioned HoneyLLMd further outperforms the LLM-only one. In particular, the maximum session duration of fully-functioned HoneyLLMd increases by 148.14 s, reaching 390.67 s, while the average duration improves from 63.64 s to 76.41 s. This implies that the LLM+HPA configuration achieves more robust performance, thereby allows it to capture a much broader range of malicious behaviors. In the meantime, the fully-functioned HoneyLLMd collects 2,842 unique commands, a substantial increase of 1,337 over the LLM-only system, suggesting that the adaptive defense strategy effectively mitigates attacker suspicion and maintains engagement far longer than both non-adaptive models.

In summary, compared with Amun and Cowrie, the two LLM-supported honeypot systems are superior with significantly longer session durations. Moreover, the HPA component effectively enhances both the deception performance and the data capture capability, as well as reduces the compromised risk,

thereby improving the system robustness and inducing more malicious activities.

2) *HPA and Blocking Mechanism Validation*: To further evaluate the feasibility of HPA and Blocking mechanism, we conduct a comparative experiment comparing HPA adaptive blocking against two baselines: anomaly-based blocking and random blocking. The experiment begins by performing a replay attack with the blocking mechanism disabled. As a result, we can get all the state transitions and the payoff pair (R_a^p, R_h^p) for each transition, where $R_a^p + R_h^p = 1$. Next, we block a certain rate (r) of transitions at a time and observe the overall honeypot benefit P to show the performance of different blocking methods. The HPA adaptive blocking method aims to block transitions based on their payoff; the anomaly-based blocking method aims to block the repeated transitions to filter the redundant attack information; the random blocking method aims to block the transitions at random.

HPA adaptive Blocking: The HPA adaptive blocking first sorts all transitions in descending order based on the value of R_a^p . The first i transitions, which are determined by the block rate r , are blocked. For these blocked transitions, their payoff values are uniformly set to the payoff of the i -th transition, denoted as α , to emulate a threshold decision. Therefore, the overall honeypot benefit for HPA adaptive blocking P_h can be calculated as follows:

$$P_h = \frac{\sum_{i=1}^{rN} (1 - \alpha) + \sum_{j=i+1}^N (1 - R_{a,j}^p)}{N}, \quad (12)$$

where N is the total number of transitions, rN represents the number of blocked transitions, and $R_{a,j}^p$ is the payoff for the j -th unblocked transition.

Anomaly-based Blocking: The anomaly-based blocking method is a baseline that simulates the blocking behavior caused by the frequency of transitions. It merely blocks the transitions based on their frequency, without taking into account the correlations between transitions, i.e., the payoff between them. Therefore, the transitions are ranked based on their frequency in descending order. Similarly, the first i transitions are blocked according to the block rate r . The R_a^p of blocked transitions are set to 0 to simulate failed attack, as transitions that occur frequently enough to be blocked are considered redundant for the honeypot system, indicating that the attack no longer provides meaningful interactions. The overall honeypot benefit P_a for anomaly-based blocking can be calculated as Formula (13).

$$P_a = \frac{\sum_{i=1}^{rN} (1 - 0) + \sum_{j=i+1}^N (1 - R_{a,j}^p)}{N} \quad (13)$$

Random Blocking: The random blocking baseline does not rely on sorting the transitions at all. Similarly, a certain percentage r of transitions are randomly chosen and blocked. The blocked transitions are assigned with $R_a^p = 0$ to indicate failed attack. As the blocked transitions might not be consecutive, the R_h^p of unblocked transitions are iteratively accessed. Let the unblocked transitions belong to set $\mathbb{R}$. The overall honeypot benefit P_r of random blocking is calculated as:

$$P_r = \frac{rN + \sum_{j \in \mathbb{R}} (1 - R_{a,j}^p)}{N}. \quad (14)$$

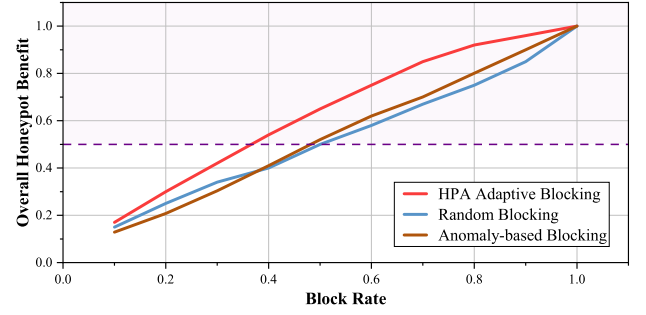


Fig. 11. Honeyplot Payoff Comparison of Blocking Methods.

TABLE VII
ATTACK INFO VARIATIONS WITHIN THE SIMULATION

Attack Purpose	Original Commands	Original Transitions	Commands Variants	Extra Transitions
CPU Info Detection	4	14	6	13
Memory Info Detection	1	5	3	8
System Info Detection	3	3	3	1
User & Thread Monitor	2	2	6	7
Binary File Inspection	2	3	3	5
Task Enumeration	1	2	3	7
Passwd+Exec Exploit	1	3	2	7
SSH Key Persistence	3	12	2	10

Fig. 11 presents the overall honeypot benefit under different block rates. The HPA adaptive blocking consistently achieves higher honeypot benefit across all block rates. In the low block-rate regime (0.1–0.3), HPA exhibits a noticeably steeper growth trend than the two baselines. For instance, at a block rate of 0.3, HPA already achieves a benefit of 0.42, compared with 0.34 for random blocking and 0.30 for anomaly-based blocking, demonstrating its early-stage efficiency. At a block rate of 0.5, HPA attains a honeypot benefit of 0.65, whereas random and anomaly-based blocking only reach 0.50 and 0.52, respectively. This indicates that HPA can obtain comparable or higher system benefit while blocking substantially fewer transitions. In particular, HPA achieves the minimal interaction goal ($R_h^p \geq 0.5$) at approximately a 0.37 block rate, whereas both baselines require a block rate close to 0.5. This represents a reduction of more than 25% in blocking cost, highlighting the efficiency gain of HPA adaptive blocking. Overall, the results demonstrate that HPA adaptive blocking not only maximizes honeypot self-benefit but also preserves richer attack interaction data, outperforming both anomaly-based and random blocking baselines.

3) *System Validation Under Cyber-Range Simulation*: A customized CTF challenge is defined for this simulation, in which the CTFAgent follows a prescribed process vector sampled from the HPA transition matrix to accomplish the target objective, i.e., establishing persistence via a malicious SSH key. During the interaction, HoneyLLMd dynamically determines blocking decisions and generates deceptive responses. As a result, the CTFAgent will try alternative strategies to finish the challenge.

Table VII summarizes the variations in attack commands and state transitions recorded by HoneyLLMd before and after the simulation with CTFAgent. The original process vector reflects eight attack purposes, such as information detection activities

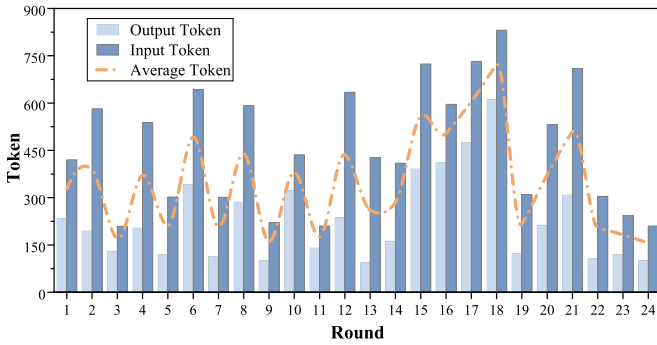


Fig. 12. Token Consuming per Round.

(e.g., CPU, memory, and system inspection), exploitation and backdoor behaviors. After the simulation, noticeable increases in commands are observed across all attack goals. Correspondingly, extra state transitions are recorded in the HoneyLLMd system. This variance is particularly pronounced in the information detection phase, where CPU information detection alone introduces 6 additional commands and 13 new transitions. Moreover, 17 extra transitions are observed in password modification and execution as well as SSH key-based persistence, indicating that the attacker have made extensive attempts to establish backdoor access. This further indicates that HoneyLLMd's block mechanism and deceptive responses lead the agent to explore additional execution paths and adjust its operational strategies.

Fig. 12 illustrates the token consumption of CTFAgent over 24 interaction rounds. Input tokens represent the contextual overhead required to perceive environmental feedback and historical interaction traces, whereas output tokens capture the computational reasoning cost associated with LLM-driven strategic decision-making and replanning. As not all commands are blocked, low token consumption is expected when the agent directly executes commands with minimal deliberation. Noticeable increases in token consumption are observed at several key interactions. During rounds 1–6, an average of approximately 400 tokens is consumed, corresponding to information detection activities. A more pronounced increase occurs in rounds 13–18, which align with credential manipulation behaviors, where token usage peaks with the highest input token count (821) and output token count (611). Token consumption also rises during rounds 19–21, corresponding to backdoor operations. Overall, the elevated input and output token usage at these critical interactions provides direct evidence that HoneyLLMd effectively induces deeper reasoning and diversified path exploration in CTFAgent, thereby demonstrating its effectiveness.

G. Discussion

The presented discussion summarizes several limitations of our current work and outlines directions for future research.

First, we currently model honeypot and attacker payoffs as perfectly complementary (their sum equals one), which may not reflect real interactions. In practice, their outcomes are not necessarily constant-sum, as both sides can incur costs or gain benefits independently, influenced by factors such as operational overhead, detection risk, and the value of collected information.

Second, our evaluation of LLM-driven deception is limited to four models for economic reasons. This constraint may bias our conclusions about temptation and engagement. To strengthen external validity, future work should broaden the model set, and report cost-performance tradeoffs (latency, token cost, and deception metrics). Such an expanded evaluation would also enable more robust statements about which model capabilities (e.g., contextual understanding, code synthesis, hallucination tendencies) most strongly drive honeypot effectiveness.

Finally, although the LLM-based shell decoder consistently achieves richer, more semantically coherent interpretations of attacker input than a rule-based RE approach, its runtime cost is substantially higher. We deliberately retained the LLM decoder in this study because its expressive power better supports mapping free-form attacker commands into the hierarchical states required by our HPA framework. However, the latency and compute overheads are nontrivial and may impede real-time deployment. A pragmatic path forward is to explore hybrid pipelines (e.g., a fast RE front end that delegates ambiguous or complex cases to an LLM), or lightweight LLMs tuned specifically for shell parsing to reduce latency while preserving semantic accuracy.

VI. CONCLUSION

This paper presented a novel approach to enhancing cybersecurity measures through the implementation of an LLM-powered adaptive honeypot. By integrating advanced LLM technology, the proposed system successfully emulates authentic shell interactions without exposing the vulnerabilities of a real OS, thereby reducing the inherent security risks associated with HIHs. Moreover, the innovative use of HPA modeling attack process for optimizing the deception defense strategy not only deceives attackers into revealing valuable information through detailed “request-response” interactions but also maintains a low risk profile. The experimental results from our proof-of-concept system have demonstrated the effectiveness of this approach, showcasing its ability to capture a wide range of real-world attacks while minimizing security risks. This balance represents a significant advancement in the field of cybersecurity, offering a solution to the ongoing challenge of engaging attackers without compromising the security of information systems.

Future research could focus on refining the LLM's understanding and response to more sophisticated cyber-attack techniques, potentially through the integration of specialized cybersecurity datasets during the training phase. Furthermore, expanding the range of emulated environments beyond shell interactions to include web applications, databases, and other critical systems could broaden the scope of attacks that can be detected and analyzed. In addition, to balance the trade-off between shell decoding accuracy and efficiency, exploring the application of a distillation technique to open-source LLMs would be valuable.

REFERENCES

- [1] L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2002.

- [2] W. Fan, Z. Du, D. Fernández, and V. A. Villagrà, "Enabling an anatomic view to investigate honeypot systems: A survey," *IEEE Syst. J.*, vol. 12, no. 4, pp. 3906–3919, Dec. 2018.
- [3] N. Provos and T. Holz, *Virtual Honeypots: From Botnet Tracking to Intrusion Detection*, 1st ed. Boston, MA, USA: Addison Wesley, Jul. 2007.
- [4] S. Wu and W. Fan, "An SDN-NFV-enabled honeypot for manipulating command control shell TCP connection," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2023, pp. 1–5.
- [5] X. Jiang and D. Xu, "Collapsar: A VM-based architecture for network attack detention center," in *Proc. USENIX Secur. Symp.*, 2004, pp. 15–28.
- [6] M. Vrabie et al., "Scalability, fidelity, and containment in the potemkin virtual honeyfarm," in *Proc. 20th ACM Symp. Operating Syst. Princ.*, Brighton, U.K., 2005, pp. 148–162.
- [7] R. A. Rodríguez-Gómez, G. Macià-Fernández, and P. García-Teodoro, "Survey and taxonomy of botnet research through life-cycle," *ACM Comput. Surv.*, vol. 45, no. 4, pp. 1–33, 2013.
- [8] E. Kasneci et al., "Chatgpt for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, 2023, Art. no. 102274.
- [9] Y. Chang et al., "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, 2024.
- [10] N. Provos, "A virtual honeypot framework," in *Proc. 13th Conf. USENIX Secur. Symp.*, Berkeley, CA, USA: USENIX Association, 2004, pp. 1–14.
- [11] S. Almotairi, A. Clark, G. Mohay, and J. Zimmermann, "A technique for detecting new attacks in low-interaction honeypot traffic," in *Proc. 4th Int. Conf. Internet Monit. Protection*, 2009, pp. 7–13.
- [12] S. Schindler, B. Schnor, S. Kiertscher, T. Scheffler, and E. Zack, "HoneydV6: A low-interaction IPv6 honeypot," in *Proc. Int. Conf. Secur. Cryptogr.*, 2013, pp. 1–12.
- [13] N. Naik, P. Jenkins, R. Cooke, and L. Yang, "Honeypots that bite back: A fuzzy technique for identifying and inhibiting fingerprinting attacks on low interaction honeypots," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2018, pp. 1–8.
- [14] N. Fitri, A. Budi, I. Kustiawan, and S. Suwono, "Low interaction honeypot as the defense mechanism against slowloris attack on the web server," in *Proc. IOP Conf. Ser., Mater. Sci. Eng.*, 2020, vol. 850, Art. no. 12037.
- [15] J. G. Göbel, "Amun: A python honeypot," Inst. Comput. Sci., Univ. Mannheim, Tech. Rep., Mannheim, Germany, 2009. [Online]. Available: <https://ub-madoc.bib.uni-mannheim.de/2595>
- [16] T. H. Project, "Dionaea - caught bugs," Nov. 2019. [Online]. Available: <http://dionaea.carnivore.it/>
- [17] M. Oosterhof, "Cowrie," 2016. [Online]. Available: <https://github.com/cowrie/cowrie>
- [18] G. Portokalidis, A. Slowinska, and H. Bos, "Argos: An emulator for fingerprinting zero-day attacks for advertised honeypots with automatic signature generation," in *Proc. 1st ACM SIGOPS/EuroSys Eur. Conf. Comput. Syst.*, New York, NY, USA: ACM, 2006, pp. 15–27.
- [19] W. Cabral, C. Valli, L. Sikos, and S. Wakeling, "Review and analysis of cowrie artefacts and their potential to be used deceptively," in *Proc. Int. Conf. Comput. Sci. Comput. Intell.*, 2019, pp. 166–171.
- [20] C. Gupta, T. Van Ede, and A. Continella, "HoneyKube: Designing and deploying a microservices-based web honeypot," in *Proc. IEEE Secur. Privacy Workshops*, 2023, pp. 1–11.
- [21] E. López-Morales et al., "HoneyPLC: A next-generation honeypot for industrial control systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 279–291.
- [22] S. Hilt, F. Maggi, C. Perine, L. Remorin, M. Rösler, and R. Vosseler, "Caught in the act: Running a realistic factoryhoneypot to capture real threats," *Trend Micro Res.*, 2020. [Online]. Available: https://documents.trendmicro.com/assets/white_papers/wp-caught-in-the-act-running-a-realistic-factory-honeypot-to-capture-real-threats.pdf
- [23] Y. Adachi and Y. Oyama, "Malware analysis system using process-level virtualization," in *Proc. IEEE Symp. Comput. Commun.*, 2009, pp. 550–556.
- [24] I. Alberdi, É. Philippe, O. Vincent, and N. M. Kaâniche, "SHARK: Spy honeypot with advanced redirection kit," in *Proc. IEEE Workshop Monit., Attack Detection Mitigation*, 2007, pp. 47–52.
- [25] M. Bailey, E. Cooke, F. Jahani, N. Provos, K. Rosaen, and D. Watson, "Data reduction for the scalable automated analysis of distributed darknet traffic," in *Proc. 5th ACM SIGCOMM Conf. Internet Meas.*, 2005, pp. 21–21.
- [26] D. Fraunholz, D. Krohmer, S. D. Anton, and H. D. Schotten, "Investigation of cyber crime conducted by abusing weak or default passwords with a medium interaction honeypot," in *Proc. Int. Conf. Cyber Secur. Protection Digit. Serv.*, 2017, pp. 1–7.
- [27] J. Franco, A. Aris, B. Canberk, and A. S. Uluagac, "A survey of honeypots and honeynets for Internet of Things, industrial Internet of Things, and cyber-physical systems," *IEEE Commun. Surv. Tut.*, vol. 23, no. 4, pp. 2351–2383, Fourthquarter 2021.
- [28] "T-pot: The all in one multi honeypot platform," 2023. Available at: <https://github.com/telekom-security/tpotce>.
- [29] T. Yadav and A. M. Rao, "Technical aspects of cyber kill chain," in *Proc. Int. Symp. Secur. Comput. Commun.*, 2015, pp. 438–452.
- [30] A. Safarkanlou, A. Sour, M. Norouzi, and S. E. H. Sardroud, "Formalizing and verification of an antivirus protection service using model checking," *Procedia Comput. Sci.*, vol. 57, pp. 1324–1331, 2015.
- [31] G. Willcox, "How to use a reverse shell in metasploit," Jun. 2020. [Online]. Available: <https://docs.metasploit.com/docs/using-metasploit/basics/how-to-use-a-reverse-shell-in-metasploit.html>
- [32] J. Zhang, L. Yang, S. Yu, and J. Ma, "A DNS tunneling detection method based on deep learning models to prevent data exfiltration," in *Proc. Int. Conf. Netw. Syst. Secur.*, 2019, pp. 520–535.
- [33] S. Saad, F. Mahmood, W. Briguglio, and H. Elmiligi, "JSLESS: A tale of a fileless javascript memory-resident malware," in *Proc. Int. Conf. Inf. Secur. Pract. Experience*, 2019, pp. 113–131.
- [34] G. Kaddissi, M. W. Hatoum, K. Samrouth, and A. El Attar, "Securing networks against ICMP covert channels: Detection and countermeasures," in *Proc. 5th IEEE Middle East North Afr. Commun. Conf.*, 2025, pp. 1–8.
- [35] C. Kiekintveld, V. Lis, and R. Pibil, "Game-theoretic foundations for the strategic use of honeypots in network security," in *Cyber Warfare: Building the Scientific Foundation*, Berlin, Germany: Springer, 2015, pp. 81–101.
- [36] G. Wägener, R. State, A. Dulaunoy, and T. Engel, "Self adaptive high interaction honeypots driven by game theory," in *Proc. Symp. Self-Stabilizing Syst.*, 2009, pp. 741–755.
- [37] M. Du, Y. Li, Q. Lu, and K. Wang, "Bayesian game based Pseudo honeypot model in social networks," in *Proc. Int. Conf. Cloud Comput. Secur.*, 2017, pp. 62–71.
- [38] K. Horák, B. Božanský, P. Tomášek, C. Kiekintveld, and C. Kamhoua, "Optimizing honeypot strategies against dynamic lateral movement using partially observable stochastic games," *Comput. Secur.*, vol. 87, 2019, Art. no. 101579.
- [39] Y. Wang, Z. Su, A. Benslimane, Q. Xu, M. Dai, and R. Li, "Collaborative honeypot defense in UAV networks: A learning-based game approach," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1963–1978, 2024.
- [40] O. Hayat, H. Otrók, and A. Youssef, "A Markov decision process model for high interaction honeypots," *Inf. Secur. J., A Glob. Perspective*, vol. 22, no. 4, pp. 159–170, 2013.
- [41] S. Tapaswi, A. Mahboob, A. S. Shukla, I. Gupta, P. Verma, and J. Dhar, "Markov chain based roaming schemes for honeypots," *Wireless Pers. Commun.*, vol. 78, no. 2, pp. 995–1010, 2014.
- [42] I. M. M. Matin and B. Rahardjo, "The use of honeypot in machine learning based on malware detection: A review," in *Proc. 8th Int. Conf. Cyber IT Service Manage.*, 2020, pp. 1–6.
- [43] Z. Zhong and W. Fan, "A honey-imprint enabled approach for resisting social engineering attacks," in *Proc. 24th Asia-Pacific Netw. Operations Manage. Symp.*, 2023, pp. 302–305.
- [44] Y. Yigit, O. K. Kinaci, T. Q. Duong, and B. Canberk, "TWINPOT: Digital twin-assisted honeypot for cyber-secure smart seaports," in *Proc. IEEE Int. Conf. Commun. Workshops*, May 2023, pp. 740–745.
- [45] P. Holgado, V. A. Villagrà, and L. Vázquez, "Real-time multistep attack prediction based on hidden MARKOV models," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 1, pp. 134–147, Jan./Feb. 2020.
- [46] Y. Li, W. Fan, and R. Luo, "Machine learning-based approach for enhancing multi-step attack prediction," in *Proc. 24th Asia-Pacific Netw. Operations Manage. Symp.*, 2023, pp. 48–53.
- [47] T. Ali and P. Kostakos, "HUNGTPT: Integrating machine learning-based anomaly detection and explainable ai with large language models (LLMs)," 2023, *arXiv:2309.16021*.
- [48] S. Dowling, M. Schukat, and E. Barrett, "Using reinforcement learning to conceal honeypot functionality," in *Proc. Mach. Learn. Knowl. Discov. Databases*, U. Brefeld, E. Curry, E. Daly, B. MacNamee, A. Marascu, F. Pinelli, M. Berlingerio, and N. Hurley, Eds., Berlin, Germany: Springer, 2019, pp. 341–355.
- [49] O. T. Taofeek, M. Alawida, A. Alabdulatif, A. E. Omolara, and O. I. Abiodun, "A cognitive deception model for generating fake documents to curb data exfiltration in networks during cyber-attacks," *IEEE Access*, vol. 10, pp. 41457–41476, 2022.
- [50] D. Fraunholz, M. Zimmermann, and H. D. Schotten, "An adaptive honeypot configuration, deployment and maintenance strategy," in *Proc. 19th Int. Conf. Adv. Commun. Technol.*, 2017, pp. 53–57.
- [51] J. Franco, A. Aris, L. Babun, and A. S. Uluagac, "S-POT: A smart honeypot framework with dynamic rule configuration for SDN," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 2818–2824.

- [52] T. Luo, Z. Xu, X. Jin, Y. Jia, and X. Ouyang, "IoT CandyJar: Toward SAN intelligent-interaction honeypot for IoT devices," *Black Hat*, vol. 1, no. 11, pp. 1–11, 2017.
- [53] C. Guan, H. Liu, G. Cao, S. Zhu, and T. La Porta, "HoneyIoT: Adaptive high-interaction honeypot for IoT devices through reinforcement learning," in *Proc. 16th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2023, pp. 49–59.
- [54] S. Chamotra and F. Barbhuiya, "Sage: An adaptive IoT honeypot with FSM-driven protocol emulation and GraphRAG-powered re-sponse generation," *ACM Trans. Internet Things*, vol. 7, no. 2, pp. 1–34, Apr. 2026.
- [55] Openai, "Chatgpt," Nov. 2022. [Online]. Available: <https://chatgpt.com>
- [56] Y. Fu, H. Peng, and T. Khot, "How does GPT obtain its ability? tracing emergent abilities of language models to their sources," *Yao Fu's Notion*, 2022. [Online]. Available: <https://yaofu.notion.site/How-does-GPT-Obtain-its-Ability-Tracing-Emergent-Abilities-of-Language-Models-to-their-Sources-b9a57ac0fc74f30a1ab9e3e36fa1dc1>
- [57] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.
- [58] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of github copilot's code contributions," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 754–768.
- [59] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 2339–2356.
- [60] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1865–1879.
- [61] P. Hu, R. Liang, and K. Chen, "DeGPT: Optimizing decompiler output with LLM," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267622140>
- [62] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proc. 31st Annu. Netw. Distrib. Syst. Secur. Symp.*, 2024.
- [63] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "LLM agents can autonomously hack websites," 2024, *arXiv:2402.06664*.
- [64] R. Fang, R. Bindu, A. Gupta, and D. Kang, "LLM agents can autonomously exploit one-day vulnerabilities," 2024, *arXiv:2404.08144*.
- [65] Y. Zhu et al., "Teams of LLM agents can exploit zero-day vulnerabilities," in *Proc. 19th Conf. Eur. Chapter Assoc. Comput. Linguistics*, vol. 1, 2026, pp. 23–35.
- [66] G. Deng et al., "PENTESTGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. USENIX Secur. Symp.*, 2024, pp. 847–864.
- [67] T. McIntosh et al., "Harnessing GPT-4 for generation of cybersecurity GRC policies: A focus on ransomware attack mitigation," *Comput. Secur.*, vol. 134, 2023, Art. no. 103424.
- [68] K. A. Akbar, S. M. Halim, Y. Hu, A. Singhal, L. Khan, and B. Thuraisingham, "Knowledge mining in cybersecurity: From attack to defense," in *Proc. Data Appl. Secur. Privacy XXXVI*, 2022, pp. 110–122.
- [69] S. Yao et al., "React: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Representations*, 2022.
- [70] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 68539–68551.
- [71] G. Mialon et al., "Augmented language models: A survey," 2023, *arXiv:2302.07842*.
- [72] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 9459–9474.
- [73] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, "An empirical study on challenges for LLM application developers," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 7, pp. 1–37, 2025.
- [74] M. B. Ozkok, B. Birinci, O. Cetin, B. Arief, and J. Hernandez-Castro, "Honeypot's best friend? Investigating ChatGPT's ability to evaluate honeypot logs," in *Proc. Eur. Interdiscipl. Cybersec. Conf.*, 2024, pp. 128–135.
- [75] F. Setianto, E. Tsani, F. Sadiq, G. Domalis, D. Tsakalidis, and P. Kostakos, "GPT-2C: A parser for honeypot logs using large pre-trained language models," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, 2021, pp. 649–653.
- [76] M. Boffa, G. Milan, L. Vassio, I. Drago, M. Mellia, and Z. B. Houidi, "Towards NLP-based processing of honeypot logs," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops*, 2022, pp. 314–321.
- [77] P. Balasubramanian, J. Seby, and P. Kostakos, "CYGENT: A cybersecurity conversational agent with log summarization powered by GPT-3," in *Proc. 3rd Int. Conf. Artif. Intell. Internet Things*, 2024, pp. 1–6.
- [78] A. Ravi and T. Surve, "Exploring applications of NLP to create deceptive sandbox environments with honeyfiles," Mar. 2023. [Online]. Available: osf.io/qpdjw_v1
- [79] D. Reti et al., "Act as a honeypot generator! An investigation into honeypot generation with large language models," in *Proc. 11th ACM Workshop Adaptive Auton. Cyber Defense*, 2024, pp. 1–12.
- [80] M. Sladić, V. Valeros, C. Catania, and S. García, "LLM in the shell: Generative honeypots," *IEEE Eur. Symp. Secur. Privacy Workshops*, pp. 430–435, 2024.
- [81] S. Johnson, R. Hassing, J. Pijpker, and R. Loves, "A modular generative honeypot shell," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience*, 2024, pp. 387–394.
- [82] J. Shi, S. Jiang, B. Xu, J. Liang, Y. Xiao, and W. Wang, "ShellGPT: Generative pre-trained transformer model for shell language understanding," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng.*, 2023, pp. 671–682.
- [83] A. Sezgin and A. Boyacı, "DecoyPot: A large language model-driven web API honeypot for realistic attacker engagement," *Comput. Secur.*, vol. 154, 2025, Art. no. 104458.
- [84] U. Raut, A. Nagarkar, C. Talnikar, M. Mokashi, and R. Sharma, "Engaging attackers with a highly interactive honeypot system using chatGPT," in *Proc. 7th Int. Conf. Comput., Commun., Control Automat.*, 2023, pp. 1–5.
- [85] Z. Wang et al., "HoneyGPT: Breaking the trilemma in honeypots with large language models," *Comput. Netw.*, p. 112223, 2026.
- [86] H. T. Otal and M. A. Canbaz, "LLM Honeypot: Leveraging large language models as advanced interactive honeypot systems," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2024, pp. 1–6.
- [87] W. Fan, Z. Yang, Y. Liu, L. Qin, and J. Liu, "HoneyLLM: A large language model-powered medium-interaction honeypot," in *Proc. 26th Int. Conf. Inf. Commun. Secur.*, 2024, pp. 253–272.
- [88] C. Guan, G. Cao, and S. Zhu, "HoneyLLM: Enabling shell honeypots with large language models," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2024, pp. 1–9.
- [89] P. Prasad, N. Girish, S. V. A. Vh, and P. K. S., "Generative AI SSH honeypot with reinforcement learning," in *Proc. IEEE 14th Int. Conf. Commun. Syst. Netw. Technol.*, 2025, pp. 770–775.
- [90] A. Safargaliyeva, A. Rüffer, and E. Vasilomanolakis, "OHRA: Dynamic multi-protocol LLM-based cyber deception," in *Proc. 30th Nordic Conf. Secure IT Syst.*, 2025, pp. 109–128.
- [91] V. D. Priya and S. S. Chakkaravarthy, "Containerized cloud-based honeypot deception for tracking attackers," *Sci. Rep.*, vol. 13, no. 1, 2023, Art. no. 1437.
- [92] A. Nlaha, "Global SSH & Telnet Honeypot logs (Cowrie)," Kaggle Dataset, 2022. [Online]. Available: <https://www.kaggle.com/datasets/nlaha1/global-ssh-and-telnet-honeypot-logs-cowrie>
- [93] Y. Zou, J. Liu, and W. Fan, "CTFAgent: An LLM-powered agent for CTF challenge solving," *J. Inf. Secur. Appl.*, vol. 96, 2026, Art. no. 104305.



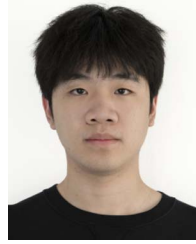
Wenjun Fan (Member, IEEE) received the Ph.D. degree in telematics systems engineering from the Technical University of Madrid, Madrid, Spain, in 2017. He is currently an Assistant Professor with the University of Washington, Tacoma, WA, USA. His research interests include cybersecurity, cloud computing, network softwarization, blockchain, and artificial intelligence. He is a member of ACM and IET.



Fei Du received the bachelor's degree in information and computing science from Xi'an Jiaotong-Liverpool University, Suzhou, China and University of Liverpool, Liverpool, U.K., in 2025. She is currently working toward the Ph.D. degree with the Department of Computer Science, University of Liverpool. Her research interests include networking, security, and artificial intelligence.



Zhihui Wang received the bachelor's degree in information and computing science from Xi'an Jiaotong-Liverpool University, Suzhou, China and University of Liverpool, Liverpool, U.K., in 2025. She is currently working toward the master's degree with the School of Engineering and Applied Science, University of Pennsylvania, Pennsylvania, PA, USA. Her research interests include network security, artificial intelligence, and the Internet of Things (IoT).



Kejing Chang received the bachelor's degree in information and computing science from Xi'an Jiaotong-Liverpool University, Suzhou, China and University of Liverpool, Liverpool, U.K., in 2025. He is currently working toward the master's degree with the School of Computing, National University of Singapore, Singapore. His research interests include system, database and information security.



Qiyu Cao is currently working toward the under-graduation degree with the School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou, China. Her research interests include human-computer interaction, artificial intelligence, and network security.



Sang-Yoon Chang received the B.S. and Ph.D. degrees from the Department of Electrical and Computer Engineering, University of Illinois Urbana-Champaign, Champaign, IL, USA, in 2007 and 2013, respectively. He was a Postdoctoral Fellow with Advanced Digital Sciences Center. He is currently an Associate Professor with Computer Science Department, University of Colorado Colorado Springs, Colorado Springs, CO, USA. His research interests include security, networking, wireless, cyber-physical systems, and applied cryptography.



Lekun Liu is currently working toward the under-graduation degree with the School of Advanced Technology, Xi'an Jiaotong-Liverpool University, Suzhou, China. His research interests include networking, security, and artificial intelligence.